



FACULTY OF INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING
DEGREE PROGRAMME IN ELECTRONICS AND COMMUNICATIONS ENGINEERING

MASTER'S THESIS

EXPERIMENTAL VALIDATION AND ANALYSIS OF THE PERFORMANCE OF DECT NR+ RADIO ACCESS TECHNOLOGY

Author	Isitha Bandara
Supervisor	Assoc. Prof. Konstantin Mikhaylov
Second Examiner	Prof. Kithsiri Liyanage
Technical Advisor	Jani Launonen

June 2025

Bandara I. (2025) Experimental validation and analysis of the performance of DECT NR+ radio access technology. University of Oulu, Faculty of Information Technology and Electrical Engineering, Degree Programme in Electronics and Communications Engineering, Master's Thesis, 79 p.

ABSTRACT

Digital Enhanced Cordless Telecommunications New Radio+ (DECT NR+) is a recently introduced non-cellular radio access technology operating in the global license-exempt 1.9 GHz band, specifically designed for dense Internet of Things (IoT) deployments. Targeting key Fifth Generation (5G) communications such as massive Machine-Type Communications (mMTC) and Ultra-Reliable Low-Latency Communications (URLLC), it promises high reliability, low latency, and extended range. However, limited empirical data exists on its practical performance.

This thesis presents a comprehensive experimental evaluation of DECT NR+ at the physical layer (PHY) using Nordic Semiconductor's nRF9161 development kits, DECT Shell (DeSh), and custom firmware. The testbed characterizes four key performance metrics: communication range, throughput, latency, and energy efficiency. Range tests conducted in both indoor and outdoor environments revealed a maximum Line-of-Sight (LoS) range of 730 meters outdoors with 80% Packet Delivery Ratio (PDR). Meanwhile at short range, maximum measured throughput was 2.498 Mbit/s at Modulation and Coding Scheme (MCS) 4 with 8 sub-slots, compared to the theoretical maximum of 3.36 Mbit/s. Latency measurements showed a minimum Round-Trip Time (RTT) of 2.255 ms for a 12-byte packet. Energy per bit ranged from 0.26 to 2.6 μJ depending on the MCS level.

Findings confirm that DECT NR+ offers significantly higher range and data rates than typical Bluetooth Low Energy (BLE) solutions but incurs higher energy costs, in both transmitter and receiver modes. These results highlight DECT NR+'s suitability for low duty-cycle, long-range, and latency-sensitive IoT applications, while identifying key areas for modem-level energy optimization.

Keywords: DECT NR+, IoT, latency, energy efficiency, communication range, throughput, nRF9161 .

Bandara I. (2025) DECT NR+ radioteknologian suorituskyvyn kokeellinen validointi ja analyysi. Oulun yliopisto, tieto- ja sähkötekniikan tiedekunta, elektroniikan ja tietoliikennetekniikan tutkinto-ohjelma. Diplomityö, 79 s.

TIIVISTELMÄ

Digital Enhanced Cordless Telecommunications New Radio+ (DECT NR+) on hiljattain esitelty soluton radioteknologia, joka toimii maailmanlaajuisesti lisenssivapaalla 1,9 GHz taajuusalueella. Se on suunniteltu erityisesti tiheisiin esineiden internetin (IoT) käyttökohteisiin. Teknologia kohdistuu viidennen sukupolven (5G) viestinnän keskeisiin osaluoiisiin, kuten massiiviseen koneiden väliseen viestintään (mMTC) ja erittäin luotettavaan matalaviiveiseen viestintään (URLLC), tarjoten korkean luotettavuuden, alhaisen viiveen ja pitkän kantaman. Kuitenkin käytännön suorituskykyä koskevaa empiiristä tietoa on vielä niukasti saatavilla.

Tämä tutkielma esittelee kattavan kokeellisen arvioinnin DECT NR+ -teknologiasta fyysisellä kerroksella (PHY) hyödyntäen Nordic Semiconductorin nRF9161-kehitysalustoja, DECT Shell (DeSh) -työkalua ja mukautettua laiteohjelmistoa. Mittausympäristössä arvioitiin neljää keskeistä suorituskykymittaria: kantama, tiedonsiirtonopeus, viive ja energiatehokkuus. Sekä sisä- että ulkotiloissa tehdyissä kantamatesteissä saavutettiin suurimmillaan 730 metrin näköyhteys (LoS) ulko-olosuhteissa, 80%:n paketinvälityssuhteella (PDR). Suurin mitattu tiedonsiirtonopeus oli 2,498 Mbit/s käyttäen MCS 4 -tasoa ja 8 aliaikaikkunaa, kun teoreettinen maksimi oli 3,36 Mbit/s. Viivetestit osoittivat pienimmäksi mitatuksi edestakaiseksi ajaksi (RTT) 2,255 ms 12 tavun paketeilla. Energiankulutus bittiä kohden vaihteli välillä 0,26–2,6 μ J riippuen MCS-tasosta. Tulokset vahvistavat, että DECT NR+ tarjoaa selvästi pidemmän kantaman ja suuremmat datanopeudet verrattuna tavanomaisiin Bluetooth Low Energy (BLE) -ratkaisuihin, mutta kuluttaa enemmän energiaa sekä lähetys- että vastaanottotiloissa. Nämä havainnot korostavat DECT NR+ -teknologian soveltuvuutta vähäkäyttöisiin, pitkän kantaman ja viiveherkkiin IoT-sovelluksiin, ja tuovat samalla esiin modeemitasoisia optimointikohteita energiatehokkuuden parantamiseksi.

Avainsanat: DECT NR+, IoT, viive, energiatehokkuus, kantama, läpäisykyky, nRF9161 .

TABLE OF CONTENTS

ABSTRACT

TIIVISTELMÄ

TABLE OF CONTENTS

FOREWORD

LIST OF ABBREVIATIONS AND SYMBOLS

1	INTRODUCTION.....	9
1.1	Background and motivation	9
1.2	Problem statement.....	9
1.3	Research objectives	10
1.4	Research scope and limitations	10
1.5	Structure of the thesis.....	10
2	RELATED WORKS	11
2.1	DECT-2020 NR link distance performance in varying environments: models and measurements	11
2.2	DECT-2020 NR for professional live audio: design space exploration and practical evaluation	12
2.3	Assessment of candidate technology ETSI: DECT-2020 New Radio	12
2.4	System reference document (SRdoc); DECT-2020 NR technology operating in frequency bands below 6 GHz	13
2.5	Examination of PHY layer features in DECT-2020 and LTE-M networks.....	13
2.6	Summary of the findings.....	13
3	TECHNICAL BACKGROUND	15
3.1	Network topologies and associated mechanisms	15
3.2	DECT-2020 NR protocol stack	16
3.3	PHY layer overview	16
3.3.1	Transmission frame structure	16
3.3.2	Modulation and Coding Scheme (MCS) levels on DECT-2020 NR.....	17
3.3.3	Transport block size (TBS)	17
3.3.4	PHY headers	18
3.3.5	Coexistence and spectrum use	18
3.3.6	Channel bands and frequency arrangement	19
3.4	MAC layer.....	19
3.5	Transmission power	20
3.6	PHY control via Nordic modem library.....	20
3.7	Nordic DECT NR+ PHY layer Shell (DeSh).....	20
3.7.1	Perf command	21
3.7.2	RSSI scanning command	21
3.7.3	Ping command.....	21
3.8	Comparison with Bluetooth Low Energy (BLE)	22
4	IMPLEMENTATION	23
4.1	Testbed setup.....	23
4.1.1	Hardware components.....	24

	4.1.2	Software components	24
4.2		Experiment design and procedures	24
	4.2.1	Maximum communication range of a single hop.....	24
	4.2.2	Maximum achievable throughput.....	26
	4.2.3	Minimum latency measurement.....	29
	4.2.4	Energy efficiency estimation	30
5		RESULTS AND DISCUSSION	32
	5.1	Maximum communication range of a single hop.....	32
	5.1.1	Test 1 : Indoor LOS	32
	5.1.2	Test 2 : Outdoor LOS.....	34
	5.1.3	Test 3 : Outdoor LOS.....	36
	5.1.4	Test 4 : Outdoor LOS.....	38
	5.1.5	Summary of range tests.....	40
	5.2	Maximum achievable throughput	40
	5.2.1	Throughput performance using Nordic's DeSh	41
	5.2.2	Throughput performance using custom firmware	41
	5.3	Minimum latency measurement.....	45
	5.4	Energy efficiency estimation.....	48
	5.4.1	TX energy per bit analysis.....	48
	5.4.2	Impact of TX power on energy consumption.....	49
	5.4.3	RX mode power consumption.....	50
6		CONCLUSION AND FUTURE WORK.....	52
	6.1	Summary of key findings	52
	6.2	Limitations	53
	6.3	Future work	53
7		REFERENCES	54
8		APPENDICES.....	56

FOREWORD

This thesis presents the experimental evaluation of DECT NR+ radio technology for low-latency and energy-efficient communication in wireless IoT systems. The research was conducted using Nordic Semiconductor's nRF9161 development kits and carried out as part of the Master's degree requirements in Wireless Communications Engineering at the University of Oulu. The work was supported by the Master's Thesis Scholarship and conducted under the guidance of faculty affiliated with the Centre for Wireless Communications (CWC). I would like to express my sincere gratitude to my supervisor, Associate Professor Konstantin Mikhaylov, for his invaluable guidance, support, and encouragement throughout the course of this thesis. I also wish to thank Professor Kithsiri Liyanage for serving as the second examiner and providing valuable feedback. I am especially thankful to Mr. Jani Launonen for his technical advice and for providing all the necessary equipment for the experiments. My sincere appreciation also goes to Professor Kari Kärkkäinen, the academic coordinator of the Wireless Communications Engineering Double Degree programme, for his consistent guidance and support throughout my Master's studies. I also extend my thanks to Gihan Wansekara for his assistance during the range testing phase. Finally, I wish to thank my family for their unwavering support, patience, and encouragement throughout my studies. Their belief in me has been a constant source of strength and motivation. Artificial intelligence applications were used to support non-content generating tasks during this thesis. ChatGPT assisted with grammar checking, proofreading, data research related to DECT NR+, and debugging custom firmware for the nRF9161 in visual studio code. All content, analysis, and results presented in this thesis are the author's original work. The correctness of all results and technical content was verified manually by the author.

Oulu, June 4, 2025

Isitha Bandara

LIST OF ABBREVIATIONS AND SYMBOLS

dB	Decibel
dBm	Decibel relative to one milliwatt
Hz	Hertz
$kbit/s$	Kilobits per second
m	Meters
mA	Milliamperes
MHz	Megahertz
ms	Milliseconds
μs	Microseconds
$\mu J/bit$	Micro-joules per bit
V	Volts
β	FFT scaling factor
μ	Subcarrier spacing factor
3GPP	3 rd Generation Partnership Project
5G	Fifth Generation
API	Application Programming Interface
BLE	Bluetooth Low Energy
BPSK	Binary Phase Shift Keying
CRC	Cyclic Redundancy Check
DeSh	Nordic's DECT Shell
DECT	Digital Enhanced Cordless Telecommunications
eMBB	Enhanced Mobile Broadband
ETSI	European Telecommunications Standards Institute
FDMA	Frequency Division Multiple Access
FEC	Forward Error Correction
FFT	Fast Fourier Transform
FSPL	Free Space Path Loss
FT	Fixed Termination
GPS	Global Positioning System
HARQ	Hybrid Automatic Repeat reQuest
ID	Identifier
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
LAN	Local Area Network
LBT	Listen-Before-Talk
LTE	Long-Term Evolution
MAC	Medium Access Control
MCS	Modulation and Coding Scheme
MIMO	Multiple-Input Multiple-Output
mMTC	massive Machine-Type Communications
NLOS	Non Line-of-Sight
NR	New Radio
OFDM	Orthogonal Frequency Division Multiplexing
PAPR	Peak-to-Average Power Ratio

PCB	Printed Circuit Board
PCC	Physical Control Channel
PDC	Physical Data Channel
PHY	Physical Layer
PPK2	Nordic's Power Profiler Kit 2
PDR	Packet Delivery Ratio
PT	Portable Termination
QAM	Quadrature Amplitude Modulation
QPSK	Quadrature Phase Shift Keying
RD	Radio Device
RF	Radio Frequency
RSSI	Received Signal Strength Indicator
RTOS	Real-Time Operating System
RTT	Round-Trip Time
RX	Receiver / Reception
SiP	System in Package
SNR	Signal-to-Noise Ratio
TBS	Transport Block Size
TDD	Time Division Duplexing
TDMA	Time Division Multiple Access
TR	Technical Report
TX	Transmitter / Transmission
UART	Universal Asynchronous Receiver / Transmitter
URLLC	Ultra-Reliable Low-Latency Communications
USB	Universal Serial Bus

1 INTRODUCTION

1.1 Background and motivation

Digital Enhanced Cordless Telecommunications (DECT) is a communication standard maintained by the European Telecommunications Standards Institute (ETSI). Originally designed to support fast roaming between networked base stations, DECT initially targeted wireless Local Area Network (LAN) applications. However, its most popular use case became single-cell cordless phones connected to traditional analog telephone lines. These cordless phones enabled users to move freely without the constraint of cables, leading to widespread adoption in homes and offices.

Over time, DECT evolved into various forms, supporting different wireless Internet-of-Things (IoT) applications such as wireless audio systems, baby monitors, and sensor networks, all operating in the license-free 1880-1900 MHz spectrum. However, with the rise of mobile phones and the decline in cordless phone usage, this DECT band has become increasingly underutilized. Although other standards were introduced for this band, including those for wide-band wireless audio transmission, and home automation, few have fully leveraged its potential.

To revitalize the DECT spectrum, ETSI released a new standard in 2020 known as DECT-2020 New Radio (NR), featuring an updated Physical layer (PHY). This standard targets Ultra-Reliable-Low-Latency Communication (URLLC) and massive Machine-Type Communications (mMTC) use cases and supports both point-to-point and mesh topologies. The commercial implementation of this protocol, known as DECT NR+, is currently available through Nordic Semiconductor's nRF91 series System-in-Package (SiP) devices, which integrate DECT NR+ alongside Long-Term Evolution (LTE) modems due to similarities in their operation. Notably, DECT NR+ operates in a globally allocated, non-cellular DECT band.

In today's world, the number of wireless devices continues to grow rapidly, placing increased demand on the radio spectrum. Modern wireless technologies must therefore support high data rates, robust operation in dense environments, and minimal communication delays. DECT NR+ aims to address these challenges by focusing primarily on providing reliable, low-latency communication in environments with high device density. Specifically, it targets two of the three key objectives of Fifth Generation (5G) NR, URLLC and mMTC rather than Enhanced Mobile Broadband (eMBB).

This research evaluates the real-world performance of DECT NR+, focusing on its PHY-layer implementation using Nordic's nRF9161 development kits. While upper-layer features such as Media Access Control (MAC) level interference and collision handling are not explored in this work, the study evaluates the peak capabilities of the PHY layer. Therefore, the presented results reflect performance in idealized conditions and may differ from application-level performance in full-stack implementations.

1.2 Problem statement

DECT NR+ has only recently become available in the commercial domain through Nordic's nRF91 series SiPs. While several theoretical analyses of the DECT-2020 New Radio (NR) standard exist, there is currently a lack of empirical data on the performance of DECT NR+ in practical scenarios. To address this gap, this thesis presents a real-world performance evaluation of DECT NR+ using the Nordic modem firmware and nRF9161 development kit.

1.3 Research objectives

This study aims to address the following four objectives:

- Evaluate the **maximum communication range** in both indoor and outdoor environments.
- Determine the **maximum achievable throughput** under PHY constraints.
- Measure the **minimum latency** using Round-Trip Time (RTT).
- Analyze the **energy efficiency**, including energy per bit and Receiver (RX) mode power consumption.

1.4 Research scope and limitations

This research is limited to point-to-point communication between two DECT NR+ nodes. The evaluation focuses exclusively on the PHY-layer and does not include MAC-layer or higher layer functionalities. Features such as mesh networking and interference management are not implemented. Custom firmware was developed for the experiments, which are designed to test edge-case performance (i.e., best-case and worst-case scenarios) based on the relevant objective. All measurements are specific to the Nordic nRF9161 development kit and the currently available DECT NR+ modem firmware provided by Nordic Semiconductor.

1.5 Structure of the thesis

This thesis is organized as follows:

- Chapter 2 reviews selected related works and publications relevant to DECT-2020 NR standard, focusing on physical layer performance and experimental evaluations.
- Chapter 3 provides technical background on the DECT-2020 NR standard and details the DECT NR+ implementation by Nordic Semiconductor, with emphasis on aspects relevant to this thesis.
- Chapter 4 describes the implementation of the experimental testbed and experimental procedures.
- Chapter 5 presents the results and discusses key findings related to performance metrics.
- Chapter 6 concludes the thesis and suggests directions for future research.

2 RELATED WORKS

Several research studies have evaluated the performance and potential of DECT-2020 NR communication technology. This chapter summarizes key experimental and analytical findings from recent publications that are relevant to the objectives of this thesis. Each section of this chapter is dedicated to a single reviewed paper, outlining its methodology, key findings, and relevance to this work. A summary of the reviewed studies is provided at the end of the chapter to consolidate the main observations. As the available literature specifically on DECT NR+ is extremely limited, the focus of this review is on studies related to DECT-2020 NR standard, which forms the foundation of DECT NR+. Only papers that investigate PHY-layer performance were considered. The reviewed works were primarily sourced from the Institute of Electrical and Electronics Engineers (IEEE) Xplore digital library. Since the DECT-2020 NR standard was introduced in 2020, all selected studies are relatively recent and reflect the current state of the technology.

2.1 DECT-2020 NR link distance performance in varying environments: models and measurements

Published in 2025 by Tampere University, this recent study [1] investigates the one-hop communication performance of DECT-2020 NR in both indoor and outdoor settings. The focus lies on analyzing Received Signal Strength Indicator (RSSI) variation, path loss, Signal-to-Noise Ratio (SNR), and packet success rate. These measurements are compared against theoretical models such as Free Space Path Loss (FSPL), 3rd Generation Partnership Project (3GPP) Technical Report (TR) 38.901, two-ray ground reflection, Okumura-Hata, and COST-231 Hata models.

The experiments employed Nordic's nRF9131 boards supporting DECT-2020 NR, equipped with vertically polarized omnidirectional PulseLarsen W5095X antennas mounted 1.5 meters above the ground. Transmitter (TX) power levels of -20 dBm, 0 dBm, and +19 dBm were tested using channels 1667 and 1677 at 1899 MHz, and Modulation and Coding Scheme (MCS) levels 1 and 4. A packet length of four slots was used, with 300 samples per point at a 1 Hz rate. Indoor distances were measured using a laser meter, while Global Positioning System (GPS) and maps were used outdoors. Outdoor environments included football fields, forests, lake-sides, sidewalks, power-line paths, and an observation tower around Tampere, Finland.

Key findings indicate that a Line-Of-Sight (LOS) range of up to 190 m was achieved at +8 dBm TX power in a factory corridor, maintaining over 90% packet delivery success. In Non-Line-Of-Sight (NLOS) scenarios, ranges exceeding 60 m were recorded at 0 dBm TX power through brick walls and cubicles. Reliable LOS communication was also demonstrated over a 40 m corridor at -20 dBm TX power. Outdoors, under power-line conditions and +19 dBm TX power, over 90% packet delivery success was maintained up to 650 m. Furthermore, communication was sustained across a lake over 2.3 km and from an observation tower up to 2.47 km, demonstrating the benefits of elevated TX and favorable terrain.

For indoor scenarios, the 3GPP TR 38.901 InH-LOS model best matched the measurements, while outdoor results showed lower path loss than Okumura-Hata and COST-231 models. Forests (NLOS) and high-traffic areas increased path loss, whereas environments with metal-coated structures introduced a tunneling effect that reduced it. RSSI and SNR values associated with over 90% packet success were observed above -90 dBm and around 11-12 dB indoors, and -95 dBm and 12-15 dB outdoors. The study concludes that DECT-2020 NR supports low-cost, battery-efficient deployments suitable for smart cities and industrial automation, achieving several hundred meters of reliable range at low TX power and modest antenna height.

2.2 DECT-2020 NR for professional live audio: design space exploration and practical evaluation

Published in 2024, this study [2] explores the feasibility of using DECT-2020 NR for professional live audio streaming, focusing on its low-latency capabilities. It discusses PHY-layer design trade-offs between latency, Radio Frequency (RF) resource usage, and reliability. Experiments were conducted using Nordic's nRF9151 development kits and DECT-2020 NR modems.

The PHY layer was implemented in the 1.9 GHz DECT band using a 1.728 MHz bandwidth. Precise time-stamping with nanosecond resolution, synchronized via Pulse-Per-Second (PPS) signals, enabled accurate latency measurements. The inter-Serial Peripheral Interface (SPI) latency, defined as the time between TX initiation and successful packet RX, was recorded across 20000 packets for each configuration, with antennas placed 30 cm apart in an open environment. Results showed a minimum measured audio latency of 2.312 ms when using a 2 sub-slot TX duration and a 1.25 ms periodicity at MCS 1. This configuration consumed 33% of RF resources. A more robust configuration yielded a latency of 2.517 ms using a 3 sub-slot TX duration at MCS 0, occupying 50% of the RF channel.

Throughput measurements under various MCS levels and TX durations showed that MCS 4 with 2 sub-slot TX and 0.21 ms periodicity achieved a throughput of 2246.4 kbit/s. The minimum PHY-layer latency was measured as 1.062 ms using 6 sub-slot periodicity at MCS 1. Attempts to lower latency below 1 ms encountered modem scheduling limitations, where the modem failed to process frequent TX requests.

The study also emphasizes that increasing TX duration or decreasing periodicity raises channel occupancy. Only certain PHY configurations met the targeted 200 kbit/s data rate with tolerable latency. Note that MAC features were not used, and audio codec or interference effects were not included. The paper concludes that DECT-2020 NR can theoretically support sub-3 ms wireless audio streaming. However, the current Nordic modem only supports a limited subset of PHY features and does not yet fulfill the strict sub-ms latency demands of professional audio.

2.3 Assessment of candidate technology ETSI: DECT-2020 New Radio

This independent evaluation [3] published in 2020, assesses DECT-2020 NR as submitted to the Radiocommunication Sector of the International Telecommunication Union (ITU-R) by ETSI. The focus is on latency, reliability, and connection density for mMTC.

Based on ITU-R reports M.2410 and M.2412, and modeling from DECT Forum's self-evaluation reports, the paper states that user-plane one-way latency targets of < 1 ms for URLLC are only achievable when Hybrid Automatic Repeat reQuest (HARQ) is disabled. Reported latencies include:

- 0.5616 ms (half slot, no HARQ)
- 0.9776 ms (full slot, no HARQ)
- 1.2272 ms (full slot, with HARQ)

It concludes that DECT-2020 NR meets latency goals only in limited configurations and does not fully meet all IMT-2020 requirements as of current evaluations.

2.4 System reference document (SRdoc); DECT-2020 NR technology operating in frequency bands below 6 GHz

The ETSI TR 103 722 [4], published by ETSI in 2024, provides a comprehensive overview of DECT-2020 NR. Section 7.1.1 of the document confirms that the technology reuses the transmission structure of classic DECT to maximize spectrum efficiency and minimize interference in the 1880-1900 MHz band.

Under open-space conditions with +19 dBm TX power and 1.728 MHz bandwidth, communication ranges of up to 6 km are reported. DECT-2020 NR supports automatic interference management via channel selection, Listen Before Talk (LBT), and power control, enabling flexible local deployments without centralized frequency planning. Table 7.1.2-1 in the document presents PHY latency metrics:

- With 27 kHz subcarrier spacing (μ), 1.728 MHz bandwidth, 416.667 (μ s) TX duration, and MCS 1 (no HARQ), a one-way user plane latency of 0.90 ms is achievable.
- With HARQ (10% retransmission), latency increases to 1.05 ms.
- Using a higher scaling factor ($\mu = 108$ kHz), 6.912 MHz bandwidth, and MCS 2, one-way latency drops to 0.22 ms with a 104.2 μ s TX duration.

2.5 Examination of PHY layer features in DECT-2020 and LTE-M networks

This study [5] compares DECT-2020 NR and LTE-M, both targeted at IoT applications. It highlights that DECT-2020 NR can achieve up to 12.47 Mbit/s using 1024 Quadrature Amplitude Modulation (QAM) with 1.728 MHz bandwidth and 27 kHz subcarrier spacing, surpassing LTE-M's maximum of 5 Mbit/s with 64-QAM. In terms of range, DECT-2020 NR is capable of achieving approximately 2.4 km range using coded Binary Phase Shift Keying (BPSK), and further extendable via mesh networking. The paper also notes that sub-10 ms latencies are achievable in multi-hop mesh topologies.

The study concludes that DECT-2020 NR is optimized for low-latency, high node-density, and local-area mesh networking, making it suitable for industrial IoT and smart city applications. While it offers higher PHY data rates and spectral efficiency than LTE-M, its direct communications range is shorter without mesh extension.

2.6 Summary of the findings

While the reviewed studies provide valuable insight into the PHY-layer performance of DECT-2020 NR, this thesis extends the state of the art by experimentally evaluating the capabilities of its latest implementation DECT NR+, using nRF9161 development kits. A summary of the key performance metrics reported in the reviewed literature is presented in Table 2.1, which serves as a benchmark for comparison with the experimental results obtained in this thesis.

Table 2.1. Summary of DECT-2020 NR performance evaluations from literature.

Section	Year	Objective / focus	Max range (m)	Min latency (ms)	Max throughput (kbit/s)	Environment	Methodology
2.1	2025	Range and propagation modeling	2470	–	–	Indoor/outdoor field	Real-world measurements based on nRF9131
2.2	2024	Low-latency streaming for pro audio	–	1.062	2246.4	Indoor lab	Custom firmware with nRF9151
2.3	2020	Latency compliance evaluation	–	0.56–1.22	–	Simulated/model-based	ITU-R aligned simulations
2.4	2024	Technical overview of PHY performance	6000	0.22–1.05	–	Open space	Spec-based modeling
2.5	2021	DECT-2020 NR vs LTE-M comparison	2400	<10 (mesh)	12470	Simulated and extrapolated	PHY modeling

3 TECHNICAL BACKGROUND

This chapter provides the technical foundation for the experimental work presented in this thesis. It begins by introducing the DECT-2020 NR standard, developed by ETSI, which serves as a modern, non-cellular radio interface targeting URLLC and mMTC in the IoT domain. Following this, the chapter focuses on DECT NR+, a proprietary implementation of the DECT-2020 NR standard by Nordic Semiconductor, tailored for their nRF91 series SiP devices. Key architectural components, protocol layers, PHY-layer structure, and configuration aspects specific to DECT NR+ are discussed in detail. These elements are essential for understanding the communication performance evaluations conducted in later chapters.

3.1 Network topologies and associated mechanisms

DECT-2020 NR supports several topologies:

- Point-to-point and point-to-multipoint: A Fixed Termination (FT) device coordinates the radio resources, while Portable Terminations (PTs) associate with it.
- Single-cell and multi-cell: One FT serves a cell, while in multi-cell setups, multiple FTs serve different areas and support roaming.
- Mesh networking: Each Radio Device (RD) can act as a FT, PT, or both. Devices can autonomously route, forward, or receive data without a central coordinator.
- Clustered tree topology: Formed when a backend-connected FT begins starts beaconing. Other RDs associate based on signal quality and routing cost.

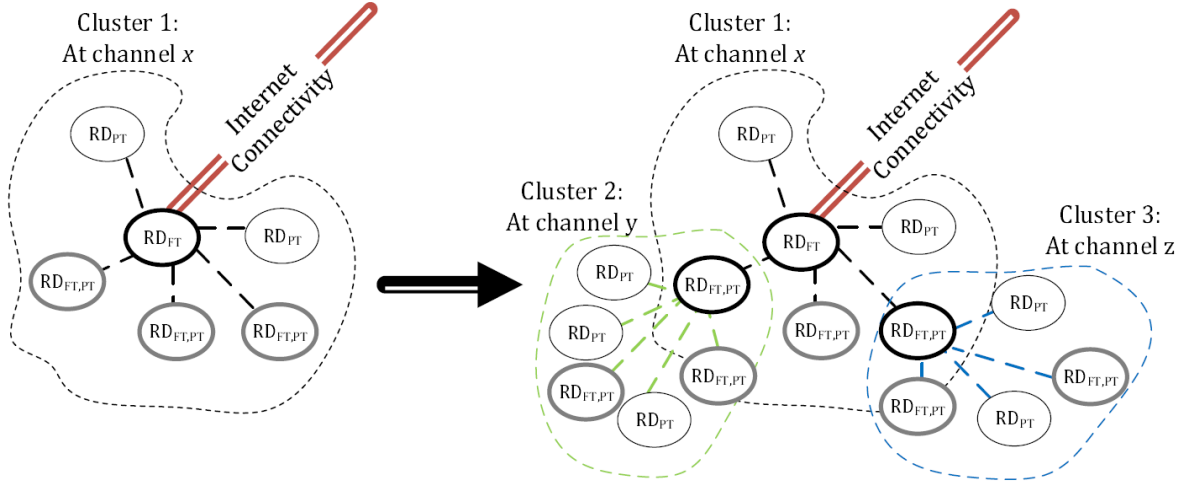


Figure 3.1. Formation of the clustered tree mesh network topology [6].

The clustered tree topology in DECT-2020 NR is formed through a distributed association process, illustrated in Figure 3.1. An RD with internet access, operating in FT mode initiates the topology by beaconing its availability and backend connectivity. This beacon includes timing, channel usage, and routing parameters. Nearby RDs evaluate the beacon and autonomously choose to associate based on the signal quality and routing cost. Once associated, these RDs may further act as intermediate nodes ($RD_{FT,PT}$), enabling multi-hop expansion of the topology. The process continues recursively, with new RDs joining based on local evaluations. The network can include multiple FT mode RDs connected to different backends, allowing flexible and scalable

mesh formation across clustered regions. This decentralized, flexible design supports dynamic and scalable deployments with autonomous routing and local link optimization.

3.2 DECT-2020 NR protocol stack

DECT-2020 NR uses a modular protocol stack [7],[8]:

- PHY-layer: Handles transmission, modulation, coding, synchronization.
- MAC-layer: Manages access, LBT, scheduling, HARQ, and power control.
- Data Link Control (DLC) layer: Enables reliable peer communication and routing.
- Convergence (CVG) layer: Maps application protocols (e.g., IPv6) to lower layers.
- Application layer: Supports user-defined applications.

3.3 PHY layer overview

The PHY layer of DECT-2020 NR handles essential signal processing operations that enable reliable wireless communication [9]. It supports subcarrier spacings ranging from 27 kHz to 216 kHz and uses scalable Fast Fourier Transform (FFT) sizes, which allow configurable RF bandwidths from 1.728 MHz up to 221.184 MHz. A wide range of modulation schemes is supported, including BPSK, Quadrature Phase Shift Keying (QPSK), 16-QAM, 64-QAM, 256-QAM, and 1024-QAM, providing flexibility to adapt to different data rate and robustness requirements.

In terms of functionality, the PHY layer performs modulation and demodulation to convert data between digital and RF domains. It employs turbo coding with a rate of 1/3 using two 8-state encoders to introduce redundancy and enhance error resilience. Forward Error Correction (FEC) and Cyclic Redundancy Check (CRC) techniques are applied to detect and correct bit errors during TX. Time and frequency synchronization processes ensure precise alignment between communicating devices. Additionally, support for advanced techniques such as Multiple-Input Multiple-Output (MIMO), beamforming, and diversity improves link quality and throughput, especially in complex radio environments.

The PHY layer also defines two main channels: the Physical Control Channel (PCC), which carries signaling and control information, and the Physical Data Channel (PDC), which transmits user payloads. These channels work together to enable coordinated and efficient data exchange across the network.

3.3.1 Transmission frame structure

As shown in Figure 3.2, a radio frame spans 10 ms and is divided into 24 slots, each 416 μs long. Depending on the subcarrier spacing factor (μ), each slot contains 10, 20, 40, or 80 OFDM symbols and is further divided into sub-slots. In this study, the default numerology parameters $\mu = 1$ and the bandwidth scaling factor $\beta = 1$ were used which corresponds to a nominal channel bandwidth of 1.728 MHz.

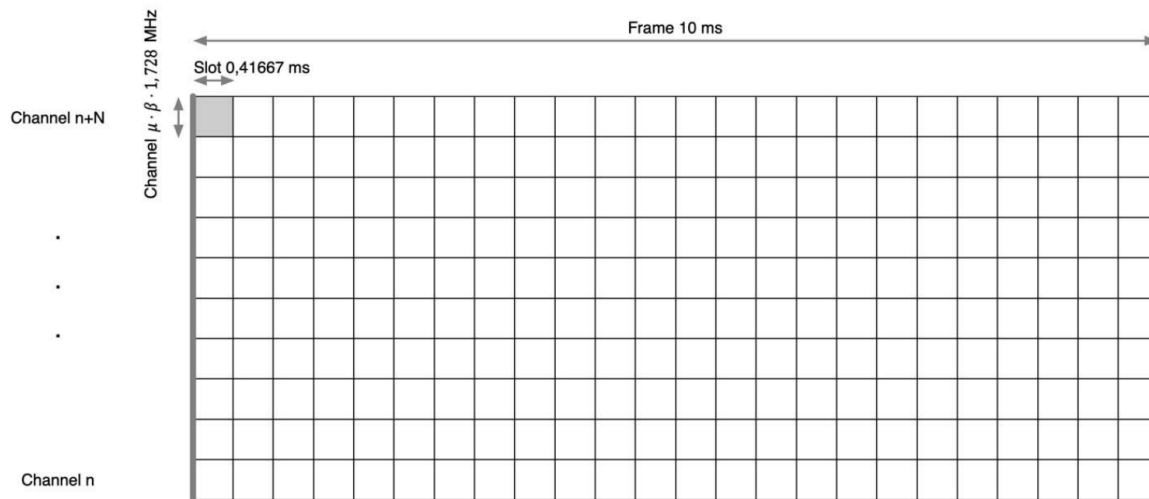


Figure 3.2. Frame structure of DECT-2020 NR[9].

3.3.2 Modulation and Coding Scheme (MCS) levels on DECT-2020 NR

Table 3.1 summarizes the modulation types and coding rates associated with each MCS level supported by DECT NR+, along with typical usage conditions. Although the DECT-2020 NR standard supports modulation schemes up to 1024-QAM, the current implementation of DECT NR+ on the nRF9161 modem supports only up to MCS level 4. Future implementations may enable DECT NR+ to achieve significantly higher throughputs by supporting higher-order modulation schemes [9].

Table 3.1. Modulation and coding rates for each MCS level [6],[9].

MCS Level	Modulation	Coding rate	Typical use case
0	BPSK	1/2	Long-range, low data rate, high robustness
1	QPSK	1/2	Balanced performance between range and speed
2	QPSK	3/4	Moderate throughput with fair reliability
3	16-QAM	1/2	Higher throughput, suitable for short-range use
4	16-QAM	3/4	Maximum throughput in favorable channel conditions

3.3.3 Transport block size (TBS)

Transmission length in DECT-2020 NR is defined in terms of sub-slots. A single transmission can span up to 15 sub-slots and the number of bits transmitted per sub-slot is based on the selected MCS. In Nordic's DECT NR+ implementation, the Transport Block Size (TBS) corresponding to each MCS and sub-slot combination is provided in the Table 3.2.

Table 3.2. Transport block size (bits) for each MCS and sub-slot combination[10].

MCS / subslot-index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
MCS 0	0	136	264	400	536	664	792	920	1064	1192	1320	1448	1576	1704	1864	1992
MCS 1	32	296	552	824	1096	1352	1608	1864	2104	2360	2616	2872	3128	3384	3704	3960
MCS 2	56	456	856	1256	1640	2024	2360	2744	3192	3576	3960	4320	4768	5152	5536	—
MCS 3	88	616	1128	1672	2168	2680	3192	3704	4256	4768	5280	—	—	—	—	—
MCS 4	144	936	1736	2488	3256	4024	4832	5600	—	—	—	—	—	—	—	—

For example, using MCS 0 with sub-slot index 1, the TBS is 136 bits, allowing for the

transmission of 17 bytes (136/8). Such a payload would require exactly 2 sub-slots. For smaller payloads, additional padding bits are appended to fill the packet structure. These TBS values represent the data payload only and exclude other physical layer overheads.

3.3.4 PHY headers

Two PHY header formats are available:

- Type 1 (40 bits): non-HARQ communication (used in this study).
- Type 2 (80 bits): Used when HARQ is enabled (this configuration was not evaluated in this study).

Table 3.3 outlines the structure of the type 1 PHY layer control field parameters, detailing each control channel field, its bit length, and functional description[7].

Table 3.3. PHY-layer control field type 1.

Control channel field	Bits	Explanation
Header Format	3	Defines the format of the control header type 1. Bits are set to 000.
Packet length type	1	Indicates whether the transmission length is indicated in subs-lots (0) or slots (1)
Packet length	4	The length of the packet transmission in sub-slots or slots.
Short Network ID	8	Short network ID of the RD
Transmitter Identity	16	Short RD ID of the RD
Transmit Power	4	Defines the used TX power
Reserved	1	-
MCS	3	Defines the MCS of the TX

3.3.5 Coexistence and spectrum use

To ensure coexistence between DECT-2020 NR systems and legacy DECT technologies, several design principles have been maintained. The smallest nominal channel bandwidth in DECT-2020 NR remains the same as that of classic DECT systems, i.e., 1.728 MHz. Wider nominal bandwidths are implemented as integer multiples of 1.728 MHz, ensuring spectral alignment. Additionally, the frame, slot, and sub-slot durations in DECT-2020 NR are harmonized with those in legacy DECT systems. Transmission durations are defined as integer multiples of sub-slot and slot lengths, further facilitating backward compatibility and spectrum coexistence. DECT-2020 NR also includes advanced spectrum management features that enhance coexistence capabilities, which include:

- TX power control, which minimizes interference by adjusting power levels based on channel conditions.
- RSSI-1 measurements, which enable the detection of any signals from other DECT systems operating on the same frequency band.
- RSSI-2 measurements, used to detect and assess transmissions from other DECT-2020 NR systems.

These mechanisms collectively support efficient operation in shared spectrum environments while minimizing interference with other DECT technologies.

3.3.6 Channel bands and frequency arrangement

The radio channel numbering scheme allows channel assignments across the frequency range of 450 MHz to 5875 MHz, organized into 17 distinct operating bands. An RD may support one or more frequency bands depending on its hardware capabilities. In this study, band 1 was used, which operates in the frequency range of 1880 MHz to 1900 MHz for both TX and RX.

According to the DECT-2020 NR channel raster, the center frequency (F_c) for bands 1-12 and 22 is calculated using the formula[11]:

$$F_c = F_0 + (n \times 0.864 \text{ MHz}) \quad (3.1)$$

Where,

- $F_0 = 450.144 \text{ MHz}$
- $n = 1, 2, 3, \dots, 2951$

In this study, channels 1665 and 1667 were used, both of which fall within band 1. Their corresponding center frequencies are:

- Channel 1665: $F_c = 1888.704 \text{ MHz}$
- Channel 1667: $F_c = 1890.432 \text{ MHz}$

The minimum channel spacing in DECT-2020 NR is 1.728 MHz. However, for bands 1-12 and 22, the step size is 0.864 MHz, resulting in a 50% spectral overlap between adjacent channels. This overlap supports denser channel reuse and is particularly advantageous in mesh and multi-link network scenarios.

3.4 MAC layer

The MAC layer in DECT-2020 NR is responsible for managing a range of control functionalities essential for establishing and maintaining radio communication. These include local radio coordination, paging, broadcasting, random access, beacon scanning, and connection configuration [7]. Additionally, the MAC layer handles channel selection, logical-to-transport channel mapping, HARQ-based error correction, and MAC-layer security through integrity protection and encryption. It also enables LBT and automatic TX power tuning to optimize communication in shared spectrum environments.

LBT is a key medium access mechanism used to reduce collisions. Before TX, an RD performs a sensing operation to determine if the channel is idle. If the channel is busy, the TX is deferred and a back-off procedure is triggered based on the MAC configuration. In Nordic's DECT NR+ implementation, as defined in the `nrf_modem_dect_phy.h` header, the LBT duration is specified as a multiple of the OFDM symbol period ($41.6 \mu s$), ranging from a minimum of 2 symbols ($83.3 \mu s$) to a maximum of 110 symbols ($4583 \mu s$). While LBT improves coexistence, it introduces a variable delay before TX, which can affect latency and throughput, particularly under heavy traffic. In this study, LBT was disabled during latency-sensitive measurements to ensure more deterministic timing.

HARQ with incremental redundancy is optionally supported in DECT-2020 NR thus DECT NR+ to improve reliability through retransmissions at the MAC layer. However, HARQ was not enabled in this work. Instead only single attempt TX communication was used to isolate and evaluate pure PHY layer performance. Key identifiers used in the MAC layer include:

- 32 bit Network Identifier (ID) - The upper 24 bits ensure global uniqueness, while the lower 8 bits help avoid local collisions.

- 32 bit Long Radio Device ID (Long RD ID) - Used for network-wide unique device identification during association, routing, and MAC-level security.
- 16 bit Short Radio Device ID (Short RD ID) - Assigned locally for PHY-layer identification and exchanged during association to link with the Long RD ID.

3.5 Transmission power

Transmit power significantly affects the communication range. Higher transmission power improves the RX signal strength, thereby increasing the link budget and extending the effective range specially under LOS conditions. The nRF9161 modem supports a set of discrete transmit power levels ranging from -12 dBm to +19 dBm. While DECT-2020 NR specification does not explicitly define an automatic power control mechanism, the DECT NR+ implementation includes the capability to perform automatic power adjustment at the MAC layer, typically based on RSSI or SNR feedback from receiver side. However, this feature was not used in the experiments conducted in this study. Instead, transmit power levels were manually configured at the PHY layer to isolate the effect of TX power on range.

3.6 PHY control via Nordic modem library

The experiments in this thesis were implemented using Nordic Semiconductor's `nrf_modem` library [12] on the nRF9161 SiP. Key Application Programming Interfaces (APIs) used include:

- `nrf_modem_dect_phy_init()` - initialize DECT PHY interface.
- `nrf_modem_dect_phy_deinit()` - de-initialize DECT PHY interface.
- `nrf_modem_dect_phy_tx()` - schedule a TX.
- `nrf_modem_dect_phy_rx()` - schedule a RX.
- `nrf_modem_dect_phy_callback_set()` - set application callbacks for PHY operations.
- `nrf_modem_dect_phy_time_get()` - retrieve 64-bit modem time (15 ns resolution).
- `nrf_modem_dect_phy_rx_stop()` - stop an ongoing RX.
- `nrf_modem_dect_phy_rssi()` - schedule an RSSI measurement operation.

TX and RX operations were controlled using `tx_op_params` and `rx_op_params`, respectively. These structures allowed configuration of network ID, PHY mode, power level, LBT status, and reception duration. The `op_complete()` callback is used to indicate the successful completion of a scheduled modem operation.

3.7 Nordic DECT NR+ PHY layer Shell (DeSh)

The DECT Shell (DeSh) is a command-line utility included in Nordic Semiconductor's DECT NR+ sample applications within the nRF connect SDK. It is built on the Zephyr RTOS and provides an interactive environment for configuring, controlling, and evaluating the DECT NR+ PHY layer on the nRF91 SiPs, particularly the nRF9161. The shell interfaces directly with the `nrf_modem_dect_phy` API and allows real-time testing of radio performance through various commands. For the purpose of this thesis, only a selected set of commands was used. These include `rssi_scan`, `perf`, and `ping`, which were employed to assess channel availability, PHY-layer throughput, and RTT latency, respectively.

3.7.1 *Perf command*

The `perf` command is used to evaluate the PHY layer throughput and link status between two devices. One board operates as the server, receiving data, while the other functions as the client, transmitting data. At the end of the test session, performance metrics such as RSSI, SNR, effective throughput, and packet error count are reported. The server is initiated by first assigning a TX ID using,

```
dect sett -t 39
```

The server is then set to RX using,

```
dect perf -s -t 15 --channel 1665
```

Here, the `-s` option sets the server mode, `-t` defines the listening duration (with `-1` used for continuous RX), and `-channel` specifies the operating frequency. On the client side, a test session is initiated using,

```
dect perf -c --s_tx_id 39 --c_tx_mcs 4 --c_slots 4 -t 15
--channel 1665
```

Where,

- `-c`: enables client mode.
- `-s_tx_id`: TX ID of the server.
- `-c_tx_mcs`: TX MCS level.
- `-c_tx_pwr`: TX power level.
- `-c_slots`: packet length in TX slots.
- `-t`: duration of the test in seconds.
- `-channel`: selected DECT NR+ channel.

This command was primarily used in maximum range experiments and for collecting key performance metrics of the DECT NR+ link.

3.7.2 *RSSI scanning command*

The `rss_scan` command performs a scan across selected channels and reports the received signal strength. It can be configured with several options, such as:

- `-c`: specifies the channel to scan.
- `-t`: sets the scan duration in milliseconds.
- `-verdict_type_count`: enables sub-slot based classification of channel usage.

An example use of this command is,

```
dect rss_scan -c 0 --verdict_type_count -t 3000
```

These scans were performed at test locations to identify usable frequency bands and avoid congested channels.

3.7.3 *Ping command*

The `ping` command measures the RTT between two nodes. One board is configured as the server, which listens and responds to ping requests, while the other acts as the client that initiates the requests. The server is started with,

```
dect ping -s --channel 1677
```

and the client is started with,

```
ping -c --s_tx_id 39 --channel 1677
```

This command was initially used as a basis for latency measurements. However, for improved timing accuracy, a custom firmware implementation was later adopted.

3.8 Comparison with Bluetooth Low Energy (BLE)

Among existing radio access technologies, Bluetooth Low Energy (BLE) is one of the closest in scope and functionality to DECT NR+, making it a relevant benchmark for performance comparison. While BLE is a mature and widely adopted protocol, DECT NR+ is a newer technology targeting similar application domains.

BLE and DECT NR+ share several characteristics:

- Operation in license-exempt bands.
- Targeting low power and short-range communication.
- Support for mesh networking architectures.
- Comparable use cases of smart sensors, industrial automation and audio streaming.

Although they differ in their PHY layer implementations, BLE operates in the 2.4 GHz band, while DECT NR+ uses OFDM based communication in the 1.9 GHz and other sub-6 GHz bands. Given their shared design objectives, BLE serves as a suitable point of reference. For this reason, DECT NR+ performance is compared with BLE in terms of communication range, data throughput, latency, and energy efficiency in Chapter 5.

4 IMPLEMENTATION

This chapter presents the implementation details of the experiments conducted. It describes the testbed used, including its hardware and software components, along with a detailed explanation of the experimental design and procedures.

4.1 Testbed setup

The testbed illustrated in Figure 4.1, was designed to facilitate the measurement of key performance indicators such as maximum communication range, maximum achievable throughput, minimum latency and energy efficiency.

To evaluate the performance of DECT NR+ radio access technology in real-world scenarios, an experimental testbed was designed using two Nordic nRF9161 development kits. These boards serve as the primary hardware platform for implementing DECT NR+ communication.

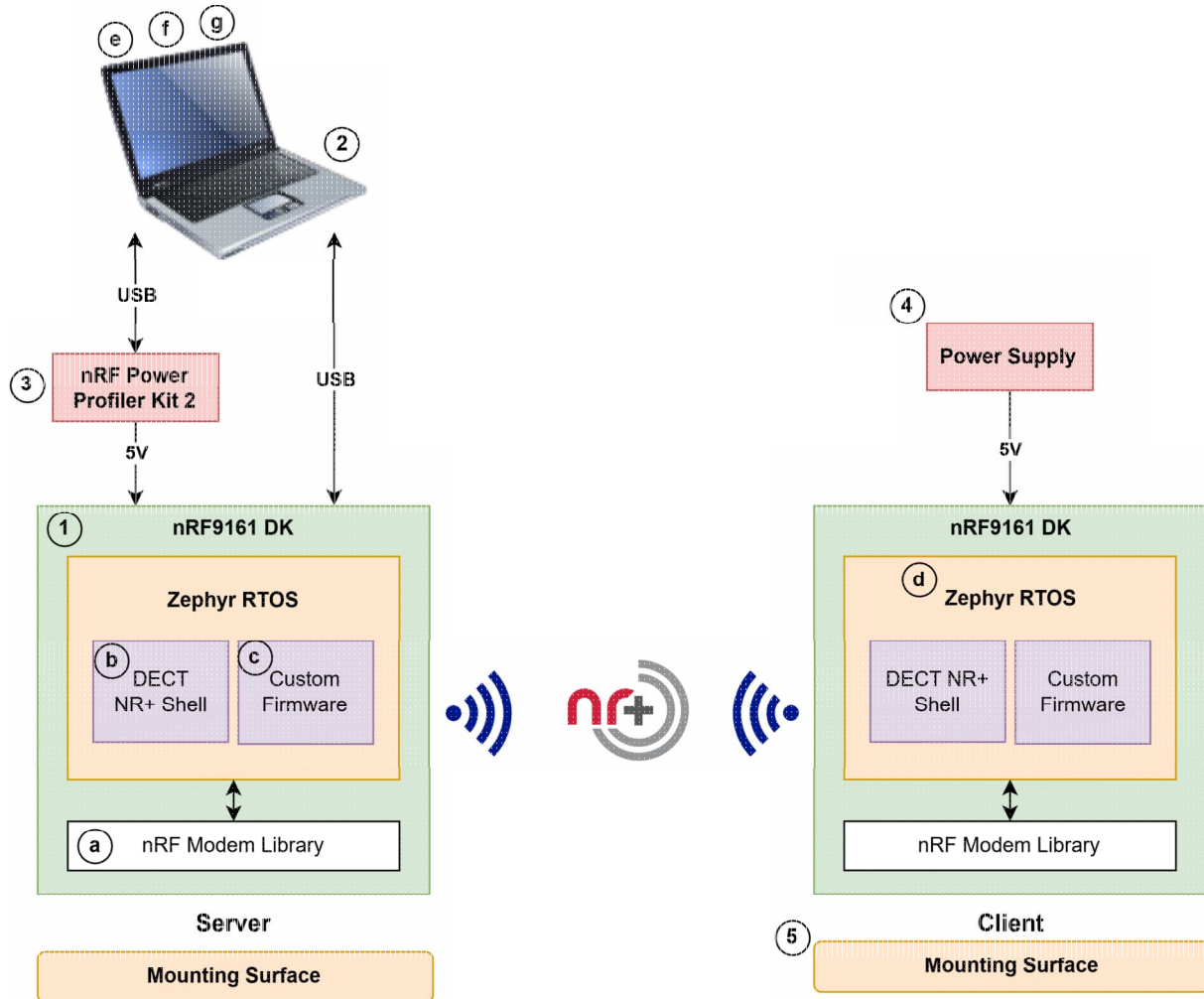


Figure 4.1. Block diagram of the DECT NR+ testbed.

4.1.1 Hardware components

1. nRF9161 development kits (2 units): Used as the TX and RX. All tests were conducted using the built-in Printed Circuit Board (PCB) antennas.
2. Laptop: Used for firmware flashing, Universal Asynchronous Receiver/Transmitter (UART) data logging and post processing. Communication with the development kits was handled via Universal Serial Bus (USB).
3. Nordic Power Profiler Kit 2 (PPK2) : Used to measure and log current consumption during experiments.
4. Power supply: The development kits were powered using either a 5V USB power bank or a laptop USB port during the experiments.
5. Mounting: During field experiments, the development kits were loosely mounted, either placed on top of a bag on the ground at an approximate height of 0.5 meters, or held alongside the laptop. All tests were performed in LOS scenarios.

4.1.2 Software components

- a. nRF Modem Library (v1.0.2): Nordic's DECT NR+ PHY Firmware used as the modem firmware for both nRF9161 development kits.
- b. DeSh (v2.9.0): Nordic's sample firmware was used as a baseline for maximum range tests.
- c. Custom Firmware: Custom applications based on DECT Hello PHY sample application were used to,
 - Implement a back to back transmission framework for throughput measurement
 - Perform RTT measurements based on echo-based communication
- d. Zephyr Real-Time Operating System (RTOS): All firmware was compiled on top of the Zephyr Operating System (OS) (v3.7.99)
- e. nRF connect for desktop (v5.1.0) : Used for programming the modem library and for configuring and logging data from the PPK2.
- f. nRF connect Software Development Kit (SDK) (v2.9.0): Development was carried out using the Visual Studio Code (VS Code) extension for nRF Connect SDK. Firmware build, flashing, and debugging were done via VS Code (v1.100.0)
- g. Logging and analysis: UART logs were collected for time-stamping and payload verification. Timing was measured using the internal timer of the nRF9161 modem.

4.2 Experiment design and procedures

This section outlines the design, configuration, and methodology used to evaluate the four key performance metrics of DECT NR+ communication using nRF9161 development kits.

4.2.1 Maximum communication range of a single hop

The goal of this experiment was to measure the maximum distance achievable by two DECT NR+ nodes as illustrated in Figure 4.2. Tests were conducted primarily in outdoor environments due to the limited availability of indoor spaces.

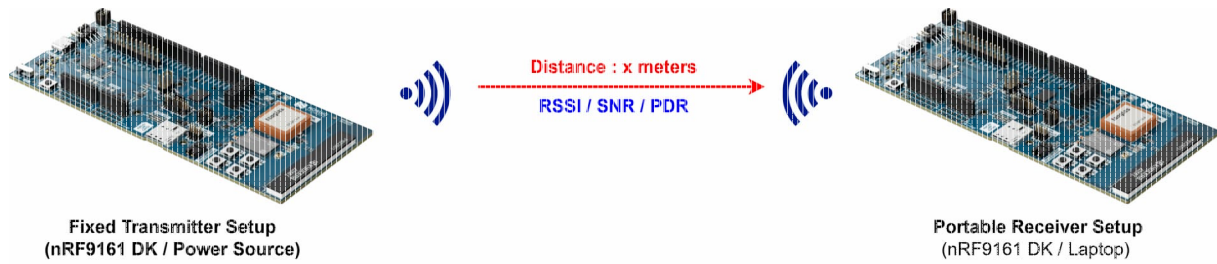


Figure 4.2. Maximum communication range experiment.

Experiment environment

All range tests were conducted in locations near the University of Oulu. The outdoor tests were performed alongside a straight section of road with an adjacent walking path, selected to ensure a favorable LOS condition, as shown in Figure 4.3. The indoor test was carried out inside a long hallway within the university premises. For the indoor case, distances were measured incrementally using a measuring tape. In the outdoor case, distances were estimated using a digital map, with local landmarks serving as reference points.

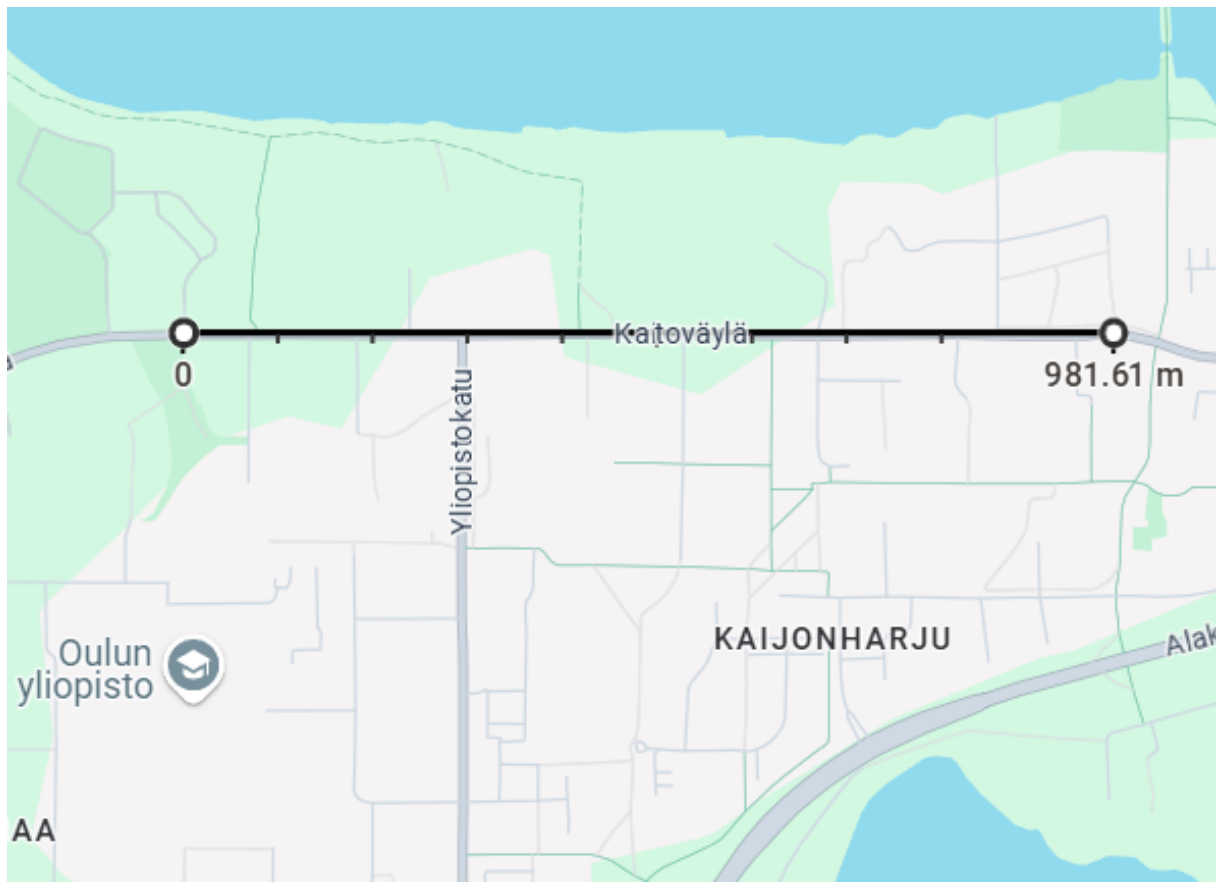


Figure 4.3. Locations near the University of Oulu where range tests were conducted.

Transmitter setup

The nRF9161 development kit was programmed with a slightly modified version of the DeSh firmware. The firmware was altered to automatically and continuously execute the `perf` com-

mand on startup. This modification was implemented within the `startup_cmd_ctrl.c` file, specifically in the `startup_cmd_worker()` function. A snippet of the modified code is provided in Annex 8.

With this change, the firmware repeatedly executes the startup command stored in memory at a defined interval. The experimental test layout is illustrated in Figure 4.2. The test was conducted under LOS conditions, where the distance between the TX and RX was gradually increased until noticeable packet loss began to occur, indicating the boundary of a reliable communication link. For the maximum range test, DECT NR+ channel 1665 was used. The `perf` client command used on the TX:

```
dect perf -c --s_tx_id 39 --c_tx_mcs 0 --c_slots 4 -t 15 --channel
1665 --c_tx_pwr 23
```

Prior to starting the experiment, the client node was connected to a laptop via USB, and the TX packet count was verified through the DeSh UART log. This packet count was then used as the reference for calculating the Packet Delivery Ratio (PDR). To ensure maximum reliability, the lowest available MCS level of MCS 0 was selected. Each packet used four slots, and the `perf` command was repeated at 10 second intervals. The test duration was set to 15 seconds, and the TX power was configured to +23 dBm; however, the output was limited to +19 dBm by hardware, which is the maximum supported output power on the nRF9161 development kit.

Receiver setup

On the RX side, an unmodified version of the DeSh firmware was used. The RX development kit was configured as the `perf` server. First, the server board was assigned a unique TX ID using the `dect sett -t 39` command. The `perf` server was then started with the following command to continuously listen for client requests:

```
dect perf -s -t -1 --channel 1665
```

The server was subsequently placed at increasing distances, while the client remained stationary. The UART output from the server was logged for post processing, enabling extraction of parameters such as received payload length, packet count, RX RSSI, SNR, and data rate. A sample UART log from the server side is shown in Figure 4.4, illustrating the output used for this analysis. Based on the number of TX and successfully RX packets, the PDR was calculated using the following equation:

$$\text{PDR (\%)} = \frac{\text{Received Packets}}{\text{Transmitted Packets}} \times 100\% \quad (4.2)$$

4.2.2 Maximum achievable throughput

This experiment aimed to evaluate the upper limit of achievable throughput using both Nordic's DeSh firmware and a custom developed application optimized to minimize TX scheduling delays. Figure 4.5 illustrates the development kit setup used for the evaluation.

Experiment environment

All throughput tests were conducted in an indoor environment, a small apartment room. The nodes were placed on a table at a fixed, short distance to minimize movement and ensure stable

```

perf server rx operation from tx id 40:
total amount of data:    4204200 bytes
packet count:           6006
elapsed time:           15.01 secs
data rates:             2240.30 kbits/secs
rx restarted count:     0
PCC CRC errors: 0
PDC CRC errors: 0
out of seqs: 0
PCC RX: MCS: 4
PCC RX: RSSI: min: -53, max: -47dBm
PCC RX: TX pwr: min 0, max 4 dBm
PCC RX: SNR: min: 87, max: 117

```

Figure 4.4. Sample UART log output from the server node used for PDR and performance metric extraction.

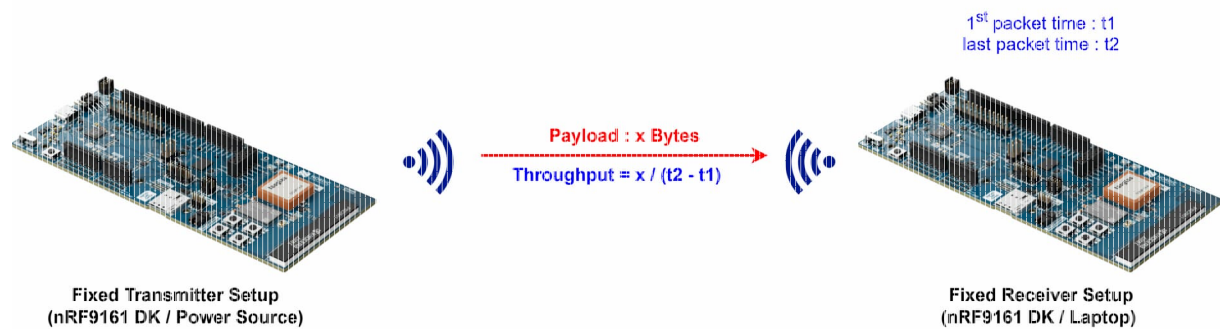


Figure 4.5. Maximum achievable throughput experiment.

operation. Both development boards were positioned with their antennas facing each other. The room contained typical household furniture and electronic devices, representing a standard indoor multi-path environment.

Throughput measurement using DeSh `perf` command

To establish a baseline, throughput measurements were first carried out using DeSh's `dect perf` command. The highest available MCS level, MCS 4 was selected to maximize the deliverable data. However, when analyzing current waveforms captured via the PPK2, it was observed that the DeSh introduces TX scheduling gaps between packets. These idle periods limit the maximum achievable throughput. To overcome this limitation and evaluate the maximum possible throughput supported by the nRF9161 modem, a custom firmware was developed.

Custom firmware for high-throughput transmission

A custom TX and RX applications were developed and implemented using the Zephyr RTOS and Nordic's DECT NR+ PHY interface. The firmware was designed to TX a fixed number of packets (N) back-to-back, with configurable MCS and sub-slot values. The total payload size and TX duration were used to compute the resulting throughput. For precise timing, modem ticks were used instead Zephyr's software based timers, which have limited resolution (on the order of milliseconds).

The DECT PHY callbacks were used to capture key metrics such as payload length and timestamp data.

Key functional steps in throughput measurement

- Initialization
 - The modem and DECT PHY interface were initialized.
 - Required data configurations for communication, such as device ID, modem callbacks registered.
- Payload setup
 - A visually recognizable character string was defined as the payload. Single character containing 1 byte of data.
 - The TX data length was selected based on Table 3.2, corresponding to the chosen MCS and sub-slot index value.
- TX sequence
 - The server TX N packets back-to-back using `nrf_modem_dect_phy_tx()`.
 - Time t_1 was recorded from the `op_complete` (operation complete) callback after the successful TX of first packet.
 - Similarly, time t_2 was recorded after the final (N^{th}) packet was transmitted.
 - Throughput was computed using N-1 packets (neglecting 1st packet) to avoid including initial delays in the measurement window.

Receiver side processing

On the RX side, modem time ticks were captured from `pdc()` callbacks. Each received packet's data length was accumulated until all N packet's had been received. The total received bytes and the timing difference $t_2 - t_1$ were used to compute the throughput. The final throughput was calculated using the equation,

$$\text{Throughput (kbit/s)} = \frac{\text{Total Received Bytes} \times 8}{\left(\frac{t_2 - t_1}{69120}\right)} \quad (4.3)$$

Here, 69120 is the modem tick rate in kHz. The result was expressed in kilobits per second (kbit/s). In addition, the TX duration was verified by measuring the power amplifier's current consumption (TX window as shown in Figure 5.11) on the nRF9161 SiP using the PPK2. This analysis was used to confirm the number of sub-slots utilized and the corresponding packet lengths.

4.2.3 Minimum latency measurement

This experiment focused on determining the lowest achievable RTT using a custom echo-based communication mechanism between two DECT NR+ nodes as illustrated in Figure 4.6. The minimum latency tests were also conducted in the same indoor environment as described in Chapter 4.2.2.

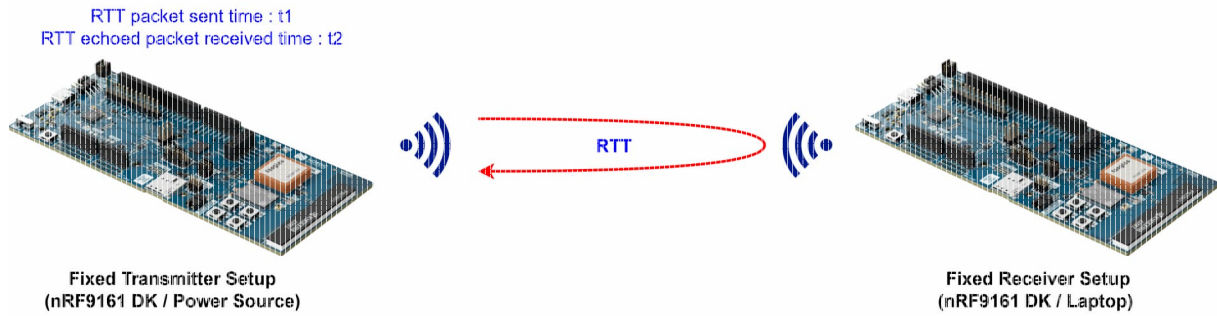


Figure 4.6. Minimum latency experiment.

To evaluate communication latency, Nordic's DeSh firmware was initially used with the ping command. However, the RTT values obtained using this method were relatively high (approximately 80 ms). To optimize for minimal latency, a custom firmware was developed. In the custom RTT measurement application, as illustrated in Figure 4.7, the client node begins by retrieving the current modem time ticks (t_1) and embedding it in the transmit buffer along with a packet ID. This packet is then sent to the server node. Immediately after TX, the client switches back to RX mode to wait for the echoed response. The server node operates as a passive echo node. Upon receiving a packet, it immediately stops the ongoing reception using the `nrf_modem_dect_phy_rx_stop()` command and transmits the same packet back without any further processing. Once the client receives the echoed packet, it extracts the original timestamp (t_1) and compares it with the timestamp at RX (t_2), which is captured using the `pdc()` callback. The RTT is then calculated as the time taken for the packet to travel from client to server and back.

To ensure reliable modem scheduling and avoid operation conflicts, a short intentional delay was introduced at server node's switching between RX to TX mode. Empirical testing revealed that a minimum delay of approximately $170 \mu\text{s}$ is required between RX stop and TX start operations at server-side. Without this delay, the modem would silently fail the subsequent immediate TX due to internal scheduling constraints.

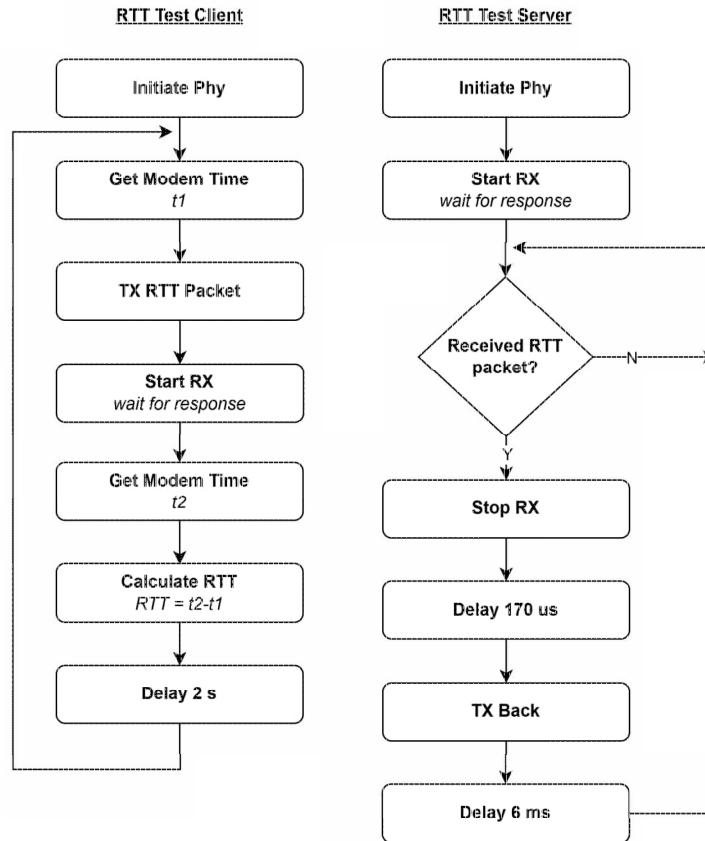


Figure 4.7. RTT measurement flow diagram.

RTT calculation formula:

$$\text{RTT (ms)} = \frac{t_2 - t_1}{69120} \quad (4.4)$$

Here, 69120 represents the modem tick rate in kilohertz (kHz), and the final RTT value is expressed in milliseconds.

4.2.4 Energy efficiency estimation

This experiment aimed to estimate the energy efficiency of DECT NR+ communications by analyzing the relationship between current consumption, TX data volume, and TX durations. The same environmental conditions described in Chapter 4.2.2 were used for these test scenarios. All power measurements were performed using the PPK2. With a sampling rate of 100 kHz, the PPK2 provides a time resolution of 10 μ s. As illustrated in Figure 4.8, the PPK2 was configured in current meter mode, receiving a 5 V input supply from the nRF9161 development kit and measuring the current drawn by the nRF9161 SiP during operation.

In addition, Peak-to-Average Power Ratio (PAPR) for different TX power levels were measured. The average current consumption on the RX side was also recorded for comparison, using the same PPK2 device connected to the RX node. Tests were carried out using various MCS levels and payload sizes. The energy consumed per bit was calculated using the equation,

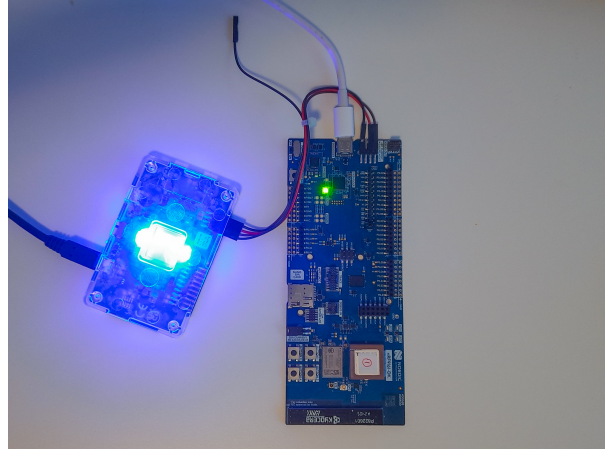


Figure 4.8. Power measurement setup using the Nordic PPK2 for evaluating current consumption during both transmission and reception in DECT NR+ operations.

$$\text{Energy per bit } (\mu\text{J}) = \frac{5 \text{ V} \times I_{\text{avg}} \text{ (A)} \times \text{TX duration } (\mu\text{s})}{\text{TBS (bytes)} \times 8} \quad (4.5)$$

Where,

- I_{avg} is the measured average current during the high-current TX window.
- TBS was determined from Table 3.2 based on the manually selected MCS/sub-slot combination used during the I_{avg} measurement.

The TX duration was measured using the current waveform profiles captured by the PPK2 and cross-validated with theoretical values based on sub-slot configurations.

5 RESULTS AND DISCUSSION

This chapter presents and discusses the experimental results obtained through the testbed implementation described in Chapter 4. The aim is to evaluate the performance of DECT NR+ communication under various real-world conditions by addressing the four main objectives outlined in this study:

- Maximum communication range of a single hop
- Maximum achievable throughput
- Minimum RTT
- Energy efficiency estimation

Each section in this chapter corresponds to one of these objectives. For each case, relevant measurements are presented along with analysis of key performance indicators such as signal strength, reliability, latency, and power consumption. The results are interpreted in the context of practical deployment considerations, and any anomalies or implementation constraints are highlighted where applicable.

5.1 Maximum communication range of a single hop

As described in Chapter 4.2.1, the TX node was placed at a fixed location while the RX was moved away in steps, and key metrics such as RSSI, SNR, PDR, and data rate at the RX were logged. For these range tests, MAC level LBT and HARQ mechanisms were not implemented. Tests were conducted under the following scenarios:

- **Test 1:** Indoor LOS, conducted in February 2025 inside a university hallway.
- **Test 2:** Outdoor LOS, performed in March 2025 at location shown in Figure 4.3.
- **Test 3 and 4:** Outdoor LOS, both conducted consecutively on the same day in April 2025 at the same location using an identical setup.

5.1.1 Test 1 : Indoor LOS

This test was conducted indoors, within the university premises, specifically along a long hallway. To gather performance measurements, Nordic's DeSh perf command was used. A TX power of +4 dBm was selected to minimize excessive signal reflections and to observe potential packet degradation over distance. The test was configured with MCS 4 and 7 sub-slots.

Table 5.1. Test 1: PDR versus distance.

Distance (m)	TX power (dBm)	Packet count	RSSI (dBm)			SNR (dB)			Packet errors	Data rate (kbit/s)	PDR (%)
			min	max	ave	min	max	ave			
3	4	6006	-82	-56	-69	9.25	29.50	19.38	26	2230.60	99.57
6	4	6006	-53	-47	-50	21.75	29.25	25.50	0	2240.30	100.00
9	4	6006	-65	-55	-60	20.50	30.25	25.38	0	2240.30	100.00
12	4	6006	-60	-52	-56	21.00	29.25	25.13	0	2240.30	100.00
15	4	6006	-78	-63	-71	11.75	30.00	20.88	0	2237.69	99.88
18	4	6006	-59	-52	-56	21.00	29.50	25.25	0	2237.69	99.88
24	4	6006	-81	-67	-74	9.00	28.00	18.50	1	2239.93	99.98
30	4	6006	-90	-69	-80	3.00	27.75	15.38	48	2222.02	99.18
37	4	6006	-72	-64	-68	14.25	28.50	21.38	0	2240.30	100.00
43	4	6006	-85	-73	-79	4.75	25.50	15.13	148	2185.09	97.54
49	4	6006	-85	-72	-79	1.75	25.25	13.50	1156	1809.10	80.75

All reported parameters are shown as recorded, except for the SNR values. In DeSh, the printed SNR is not scaled to match the Q13.2 fixed point format, which is used to represent SNR

in decibels. Therefore, during post-processing, the raw SNR values were divided by 4 to obtain the correct SNR values in dB. The variation of RSSI, SNR, and PDR with distance is illustrated in the Figures 5.1 and 5.2.

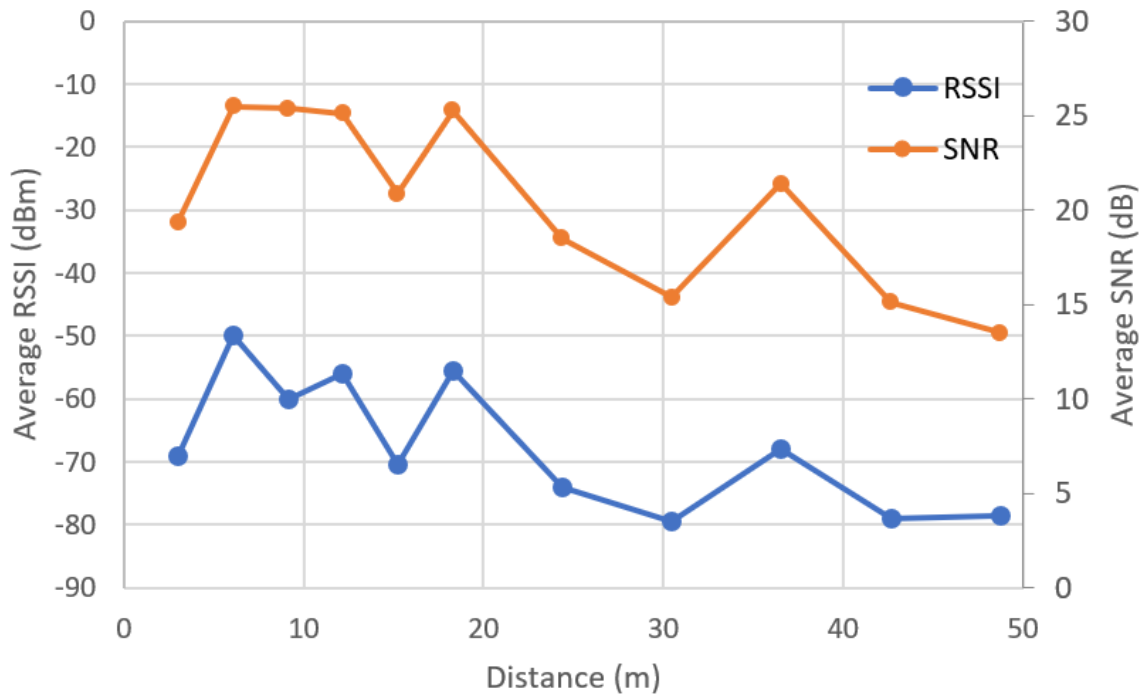


Figure 5.1. Test 1: RSSI and SNR versus distance.

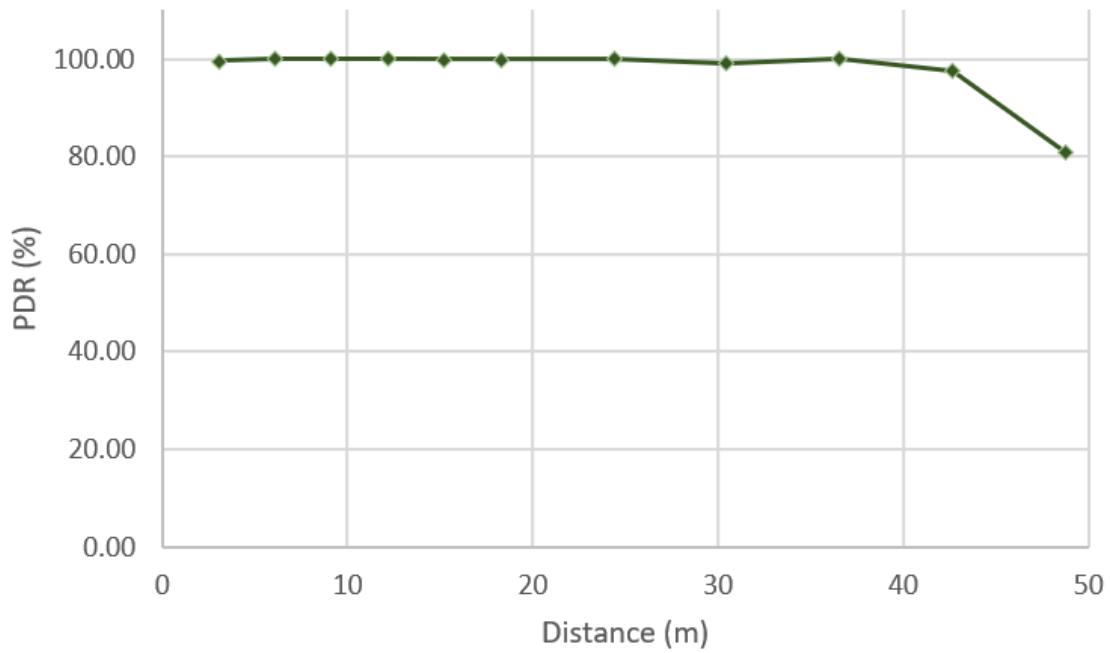


Figure 5.2. Test 1: PDR versus distance.

Upon analyzing the results, a clear correlation between RSSI and SNR values is observed. Irregular fluctuations in the RSSI curve suggest the presence of multi-path fading. The PDR dropped to approximately 80 % at a distance of around 50 meters. As illustrated in Figures 5.1 and 5.2, the DECT NR+ receiver requires average RSSI and SNR values above approximately -68 dBm and 21.38 dB, respectively, to achieve 100% PDR at MCS 4. To maintain at least 80% PDR, the corresponding thresholds are around -79 dBm for RSSI and 13.5 dB for SNR. Slightly lower PDR values at distances such as 15 meters and 18 meters, despite no packet errors, are due to packets being discarded after successful RX because they were received out of sequence.

5.1.2 Test 2 : Outdoor LOS

In this test case, the same hardware and firmware as the earlier test was used. To achieve the maximum possible range, the TX was set to the maximum TX power of +19 dBm, and the most robust MCS 0 was selected for highest reliability. The number of sub-slots used was 8.

Table 5.2. Test 2: PDR versus distance.

Distance (m)	TX power (dBm)	Packet count	RSSI (dBm)			SNR (dB)			Packet errors	Data rate (kbit/s)	PDR (%)
			min	max	ave	min	max	ave			
37	23	6006	-85	-54	-70	5.5	28.3	17	0	368.05	100.00
68	23	6006	-59	-52	-56	21.5	26.8	24	0	368.05	100.00
80	23	6006	-68	-61	-65	18.3	28.8	24	0	368.05	100.00
101	23	6006	-70	-60	-65	19.8	26.8	23	0	368.05	100.00
145	23	6006	-84	-68	-76	14.8	26.8	21	0	368.05	100.00
200	23	6006	-81	-75	-78	14.3	26.8	21	0	368.05	100.00
250	23	6006	-90	-69	-80	10.5	26.5	19	0	368.05	100.00
296	23	6006	-82	-76	-79	14.0	27.3	21	0	368.05	100.00
344	23	6006	-83	-76	-80	14.0	26.8	20	0	368.05	100.00
396	23	6006	-98	-88	-93	0.25	23.5	12	0	368.05	100.00
460	23	6006	-92	-82	-87	7.5	25.5	17	0	368.05	100.00
530	23	6006	-95	-80	-88	6.0	26.3	16	0	368.05	100.00
542	23	6006	-102	-87	-95	1.5	24.3	13	2	367.93	99.97
580	23	6006	-93	-83	-88	4.5	24.8	15	0	368.05	100.00
617	23	6006	-100	-86	-93	1.0	23.5	12	19	366.88	99.68
655	23	6006	-93	-86	-90	1.5	24.5	13	0	368.05	100.00
730	23	6006	-105	-89	-97	-5.75	18.5	6.0	919	293.41	79.72
760	23	6006	-109	-89	-99	-5.75	17.8	6.0	5476	12.21	76.06

As presented in Table 5.1.2, a high number of packet errors was recorded at the maximum detectable distance of 760 meters. These errors include both PCC and PDC RX failures. Notably, although only 6006 packets were transmitted, the RX logged 4568 successfully received packets and 5476 packet errors, exceeding the total transmitted packet count. This anomaly occurred because the RX remained active for approximately 300 seconds, waiting for the test to complete, while each TX session lasted only 15 seconds, followed by a 10 second idle period before the next session began. As a result, the RX captured packets from multiple TX sessions, many of which were either corrupted or only partially received. Additionally, it is suspected that the DeSh perf command was designed to wait until the expected number of packets had been received, which may have contributed to this issue. Further investigation is required to determine the exact cause of this behavior. Despite this, DECT NR+ demonstrated excellent long-range capability in this scenario, maintaining a PDR greater than 80% up to approximately 730 meters under clear LOS conditions, with MCS 0 and +19 dBm TX power.

As illustrated in the Figures 5.3 and 5.4, the link remained stable even as channel conditions deteriorated. For example, at around 400 meters, the average RSSI and SNR values dropped to -93 dBm and 12 dB, yet the system managed to successfully deliver all packets, maintaining a 100% PDR at that distance. Beyond 650 meters distance, as the LOS path became partially

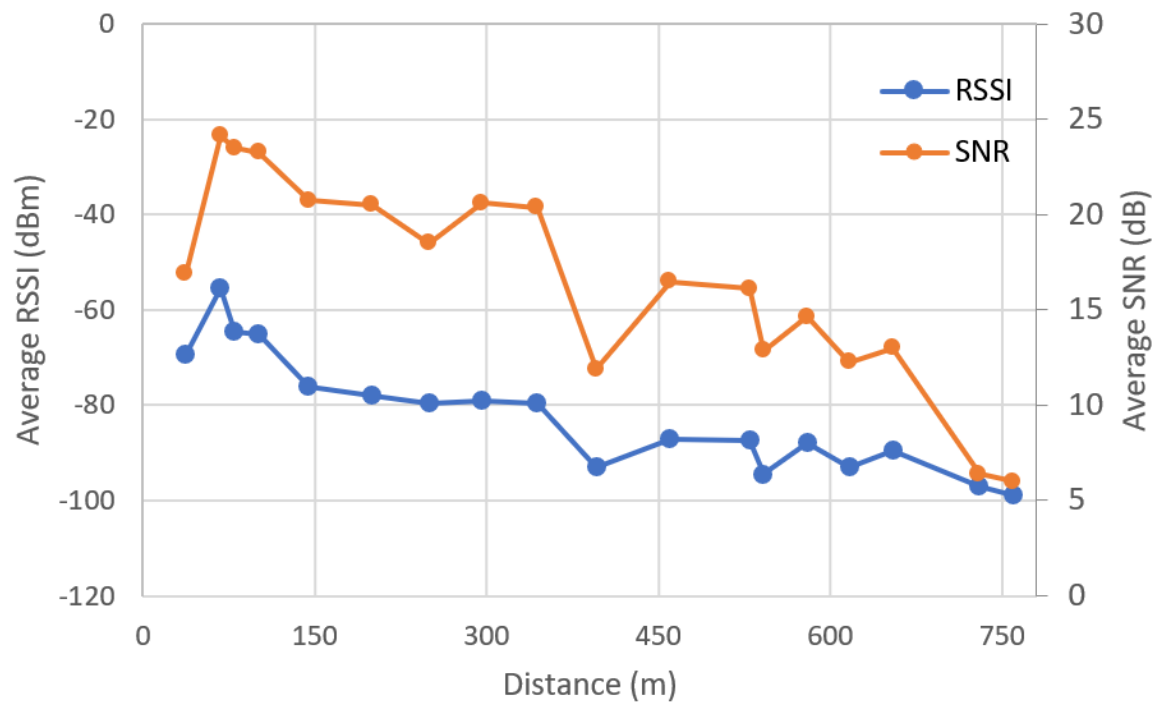


Figure 5.3. Test 2: RSSI and SNR versus distance.

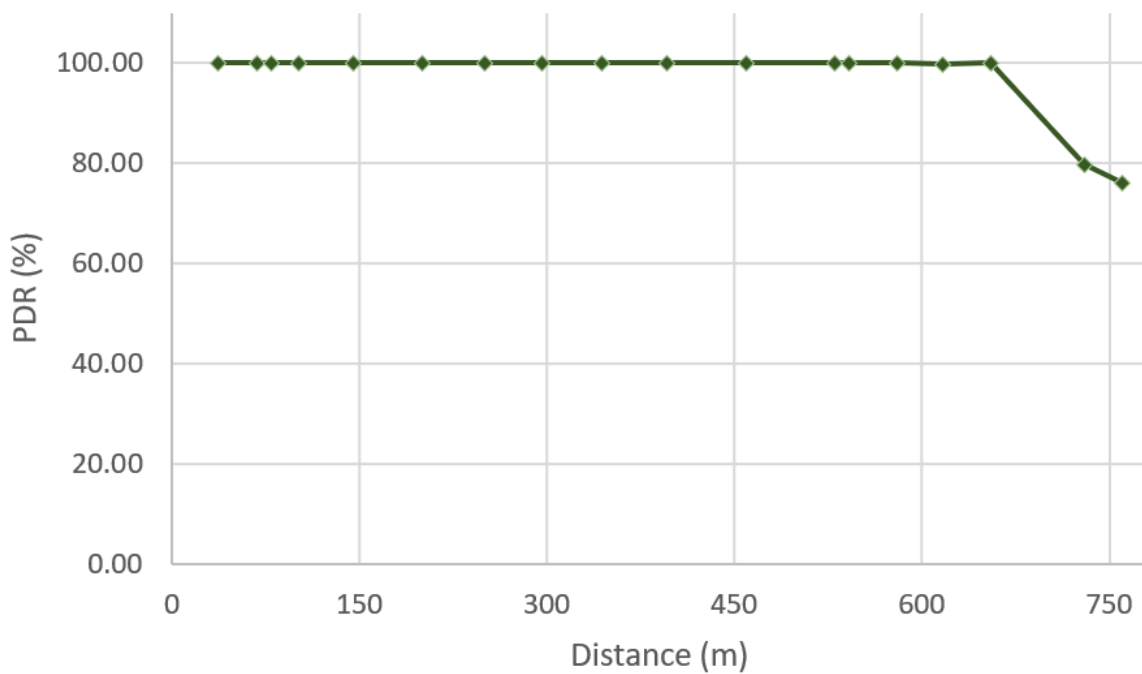


Figure 5.4. Test 2: PDR versus distance.

obstructed, the PDR dropped sharply. Although further range may have been technically possible, the performance at the 760 meter mark was likely limited by transient obstructions such as people or nearby vehicles, which appeared to significantly impact reception quality. However, in comparison to the MCS 4 configuration used in Test 1, the DECT NR+ RX demonstrated significantly more robust performance at MSC 0. Specifically, it was able to maintain a 100% PDR down to an average RSSI and SNR values of -90 dBm and 13 dB, highlighting the reliability of MCS 0 in low SNR conditions. Furthermore, an average RSSI and SNR values of approximately -97 dBm and 6 dB still achieved an estimated PDR of 80%. Notably, the PDR did not degrade gradually, but instead collapsed sharply beyond the range limit. Even minor NLOS conditions (e.g trees) at this distance caused the link to fail completely, with the receiver unable to detect any packets.

5.1.3 Test 3 : Outdoor LOS

This test was conducted with the intention of extending the range even further, building on the results of Test 2 outdoor LOS results. The same parameters were used, TX power +19 dBm, MCS 0, and 8 sub-slots.

Table 5.3. Test 3: PDR versus distance.

Distance (m)	TX power (dBm)	Packet count	RSSI (dBm)			SNR (dB)			Packet errors	Data rate (kbit/s)	PDR (%)
			min	max	ave	min	max	ave			
35	23	6006	-92	-57	-75	6.75	29.0	17.88	2	367.90	99.97
75	23	6006	-78	-67	-73	6.75	28.0	17.38	0	368.05	100.00
145	23	6006	-85	-76	-81	7.75	26.5	17.13	0	368.05	100.00
190	23	6006	-95	-82	-89	-0.25	20.75	10.25	80	363.12	98.67
260	23	6006	-92	-81	-87	1.00	22.25	11.63	19	366.86	99.68
306	23	6006	-96	-85	-91	0.00	25.25	12.63	0	368.05	100.00
320	23	6006	-101	-88	-95	-5.50	22.75	8.63	816	315.78	85.80
325	23	6006	-99	-85	-92	-1.75	20.75	9.50	304	349.15	94.87
330	23	6006	-109	-86	-98	-15.75	20.25	2.25	260	351.99	95.64
341	23	6006	-97	-85	-91	-1.25	21.75	10.25	24	366.58	99.60
343	23	6006	-92	-86	-89	2.25	20.75	11.50	0	368.05	100.00
347	23	6006	-98	-85	-92	-2.50	21.00	9.25	138	359.57	97.70
350	23	6006	-99	-87	-93	-3.25	20.25	8.50	287	350.63	95.22
368	23	6006	-102	-87	-95	-3.25	19.00	7.88	2296	217.53	59.02
380	23	6006	-100	-88	-94	-7.75	18.50	5.38	2576	210.44	57.09
388	23	6006	-98	-88	-94	-7.50	20.00	6.25	1571	269.02	73.09
390	23	6006	-102	-86	-94	-8.50	21.25	6.38	481	341.18	91.91
395	23	6006	-100	-86	-93	-4.25	21.75	8.75	311	348.93	94.81

However, the outcome was unexpectedly poor. Contrary to expectations, the maximum range performance in this case was significantly lower than in Test 2. As illustrated in the Figures 5.5 and 5.6, RSSI, SNR, and PDR values dropped more steeply with distance. In order to better understand the link degradation, additional measurements were taken at shorter gaps to observe more granular variation. A 100% PDR was achieved at an average RSSI and SNR values of -89 dBm and 11.5 dB, while approximately 80% PDR was observed around -95 dBm RSSI and 8.63 dB SNR. Figure 5.7 compares the average RSSI and SNR values between Test 2 and Test 3. It is evident that both RSSI and SNR values in Test 3 were consistently lower than in Test 2, suggesting that environmental factors, such as interference or path loss, negatively affected the performance of Test 3 from the outset.

The higher RSSI values observed in Test 2 contributed to its elevated SNR levels, indicating better signal quality and enabling reliable communication over longer distances. Notably, at the 400 meter mark, the RSSI values of both tests began to converge, and a sharp decline in Test 2's SNR was observed. Nevertheless, the SNR remained within the RX operational threshold, allowing the link to extend further in range. However, the poor performance in Test 3, prompted

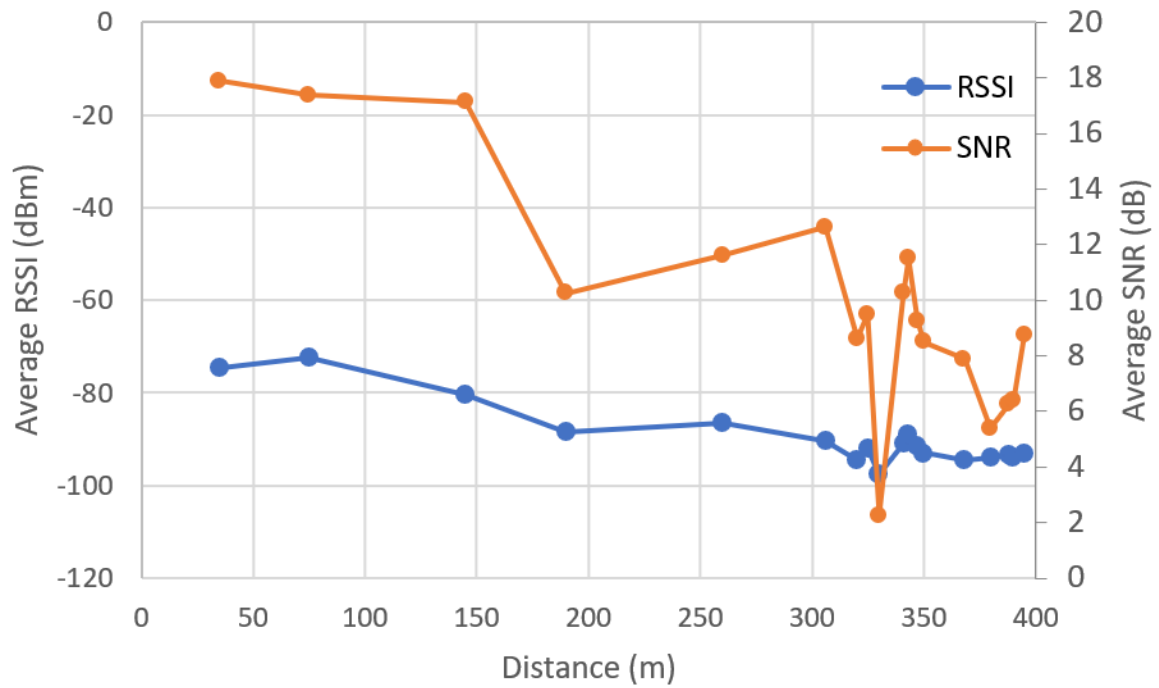


Figure 5.5. Test 3: RSSI and SNR versus distance.

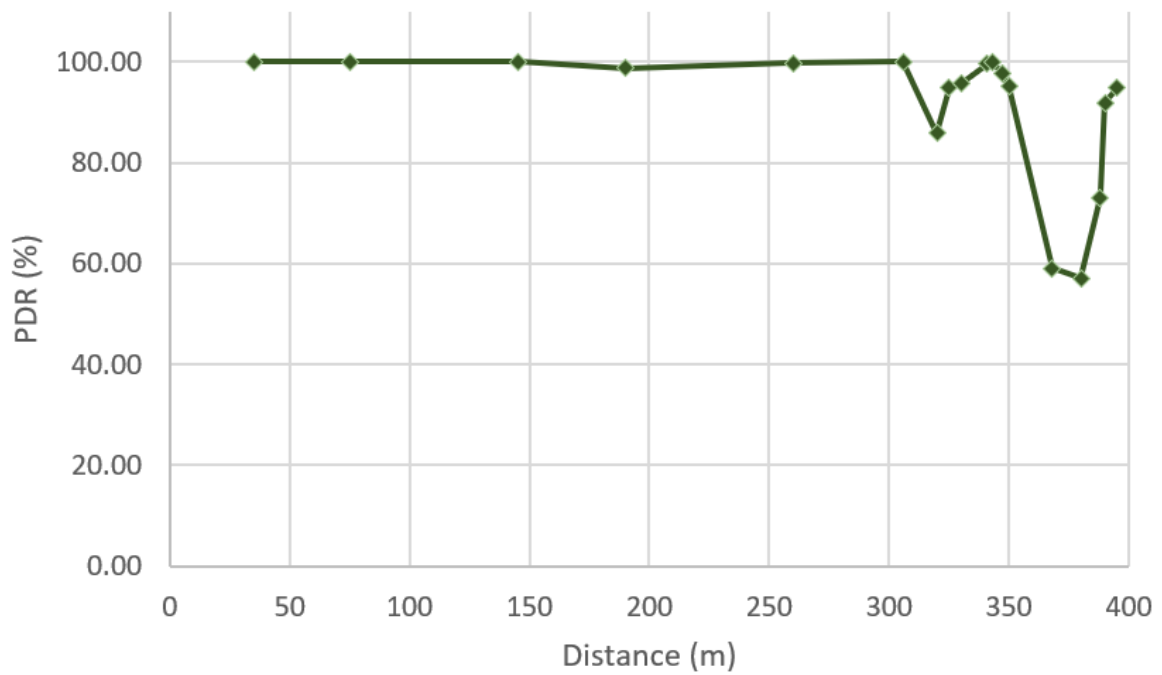


Figure 5.6. Test 3: PDR versus distance.

a follow-up test (Test 4) in the same environment to verify the results.

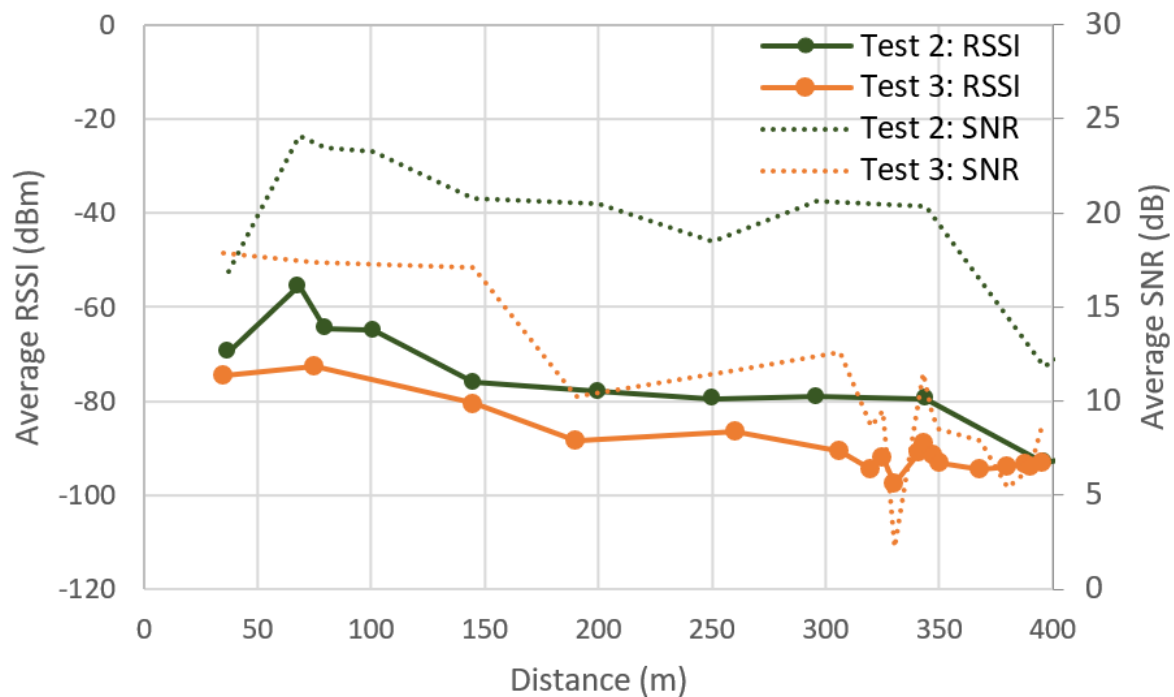


Figure 5.7. RSSI and SNR comparison between Test 2 and Test 3.

5.1.4 Test 4 : Outdoor LOS

As a follow-up to Test 3, this test was conducted using the same parameters and same environment, as outlined in Chapter 5.1.3, to validate the previously observed degradation.

Table 5.4. Test 4: PDR versus distance.

Distance (m)	TX power (dBm)	Packet count	RSSI (dBm)			SNR (dB)			Packet errors	Data rate (kbit/s)	PDR (%)
			min	max	ave	min	max	ave			
35	23	6006	-81	-65	-73	15.00	28.75	21.88	0	368.05	100.00
75	23	6006	-76	-70	-73	16.75	29.00	22.88	0	368.02	100.00
145	23	6006	-98	-68	-83	-1.00	27.75	13.38	14	367.07	99.73
190	23	6006	-96	-82	-89	-2.00	22.75	10.38	118	360.79	98.04
260	23	6006	-94	-79	-87	1.00	24.50	12.75	1	367.99	99.98
306	23	6006	-94	-82	-88	-0.75	25.75	12.50	0	368.05	100.00
320	23	6006	-97	-86	-92	-2.25	20.25	9.00	156	358.49	97.40
325	23	6006	-96	-84	-90	0.50	22.75	11.63	5	367.74	99.92
335	23	6006	-98	-87	-93	-3.50	20.75	8.63	163	358.73	97.29
341	23	6006	-100	-84	-92	-4.50	23.50	9.50	8	367.56	99.87
343	23	6006	-96	-80	-88	0.50	24.00	12.25	19	366.88	99.68
350	23	6006	-102	-86	-94	-4.75	19.75	7.50	897	312.11	84.72
368	23	6006	-102	-83	-93	-5.00	21.25	8.13	1228	292.43	79.45
380	23	6006	-98	-88	-93	-5.50	18.00	6.25	1114	299.82	81.45
388	23	6006	-97	-84	-91	-2.50	20.25	8.88	90	362.53	98.50
390	23	6006	-100	-86	-93	-4.00	22.75	9.38	256	351.63	95.54

Table 5.1.4, presents results that are consistent with those of Test 3, confirming that the earlier measurements were not due to error. However, the significantly reduced range when compared to Test 2 suggests that environmental factors may have influenced the outcome. While the exact cause remains unclear, the findings indicate that site-specific conditions, such as ambient noise, temperature variations, or transient obstructions likely played a role in limiting the effective

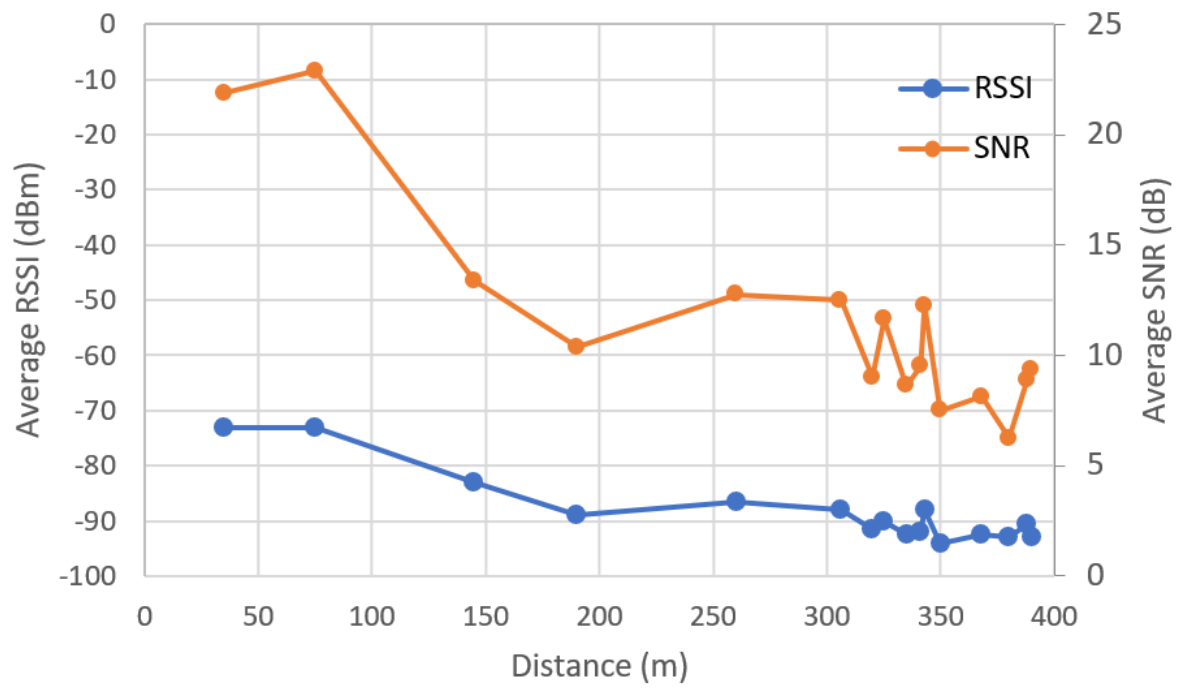


Figure 5.8. Test 4: RSSI and SNR versus distance.

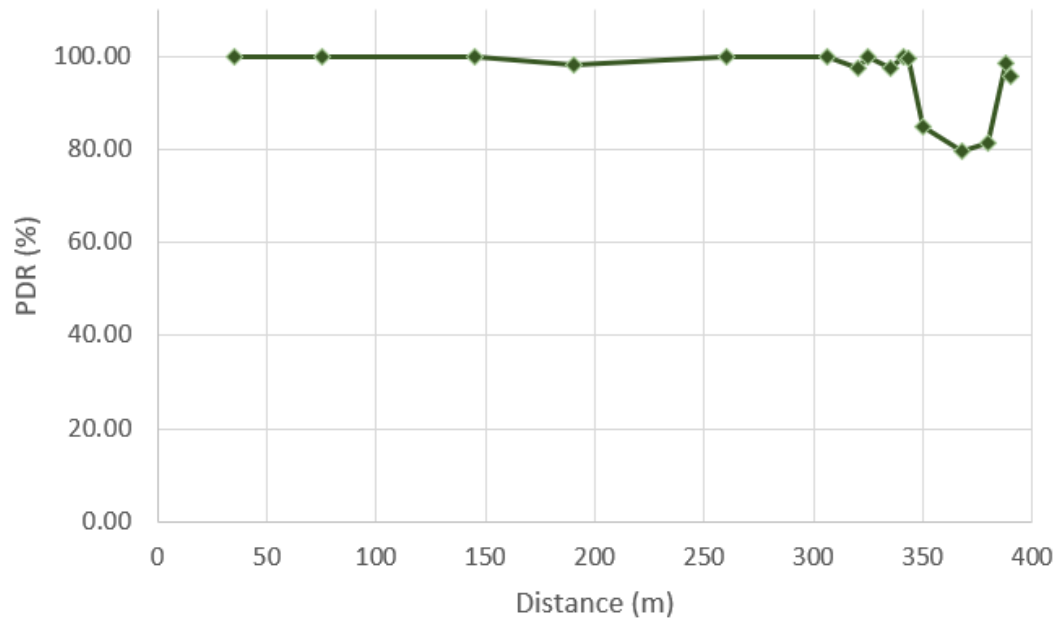


Figure 5.9. Test 4: PDR versus distance.

communication range. It is also worth noting that Test 2 was conducted under snowy conditions, whereas Test 3 and 4 were performed in a snow-free environment. This change in ground conditions may have affected signal propagation characteristics. All three outdoor tests were conducted in the same environment, along a walking pathway next to a road with buildings and trees located on either side.

5.1.5 Summary of range tests

When comparing the results from all four communication range tests, it is evident that DECT NR+ is capable of achieving extended distances, up to several hundred meters under favorable configurations. The corresponding RX-side parameters from all tests are summarized in Table 5.5.

Table 5.5. Comparison of RSSI, SNR, and max distance at 100% and 80% PDR under different test cases.

Test	100% PDR			80% PDR			MCS / TX power
	Ave. RSSI (dBm)	Ave. SNR (dB)	Max dist.	RSSI	SNR	Max dist.	
1	-68	21.38	37	-79	13.50	49	MCS 4, +4 dBm
2	-90	12.00	655	-97	6.00	730	MCS 1, +19 dBm
3	-89	11.50	343	-95	8.63	–	MCS 1, +19 dBm
4	-88	12.50	306	-93	6.25	–	MCS 1, +19 dBm

As described in the previous test cases, the PDR in Test 3 and 4 did not degrade gradually but instead collapsed abruptly. In both cases, the link was maintained up to approximately 400 meters, where the PDR was still around 95% before the connection was lost entirely. Since the PDR did not reach the 80% threshold, these results suggest that the DECT NR+ receiver requires average RSSI and SNR values above -68 dBm and 21.38 dB, respectively, to achieve 100% PDR at MCS 4. Similarly, for MCS 0, average values of approximately -89 dBm RSSI and 12 dB SNR are sufficient to sustain 100% PDR.

While the test results presented in this study are not sufficient to definitively establish the maximum achievable range of DECT NR+, literature discussed in Chapter 2 indicates that significantly longer ranges are possible under favorable conditions. For instance, ETSI's report [4] states ranges up to 6 km using +19 dBm TX power and a 1.728 MHz bandwidth. Another study [1] demonstrates a maximum range of 2.47 km with a 90% PDR using external antennas in an elevated interference-free environment, where the RX maintained as RSSI of approximately -95 dBm and SNR between 12-15 dB. A separate theoretical analysis [5] also estimates a maximum range of 2.4 km using MCS 1.

It should be noted that these range values are highly dependent on environmental conditions, antenna types, and device mounting height. In this study, the maximum observed range was approximately 730 meters with an 80% PDR using the built-in antennas on the nRF9161 development kits. Compared to BLE 5.0, which typically achieves LOS ranges of 200-400 meters [13], DECT NR+ demonstrates a notably higher communication range, particularly in the outdoor scenario of Test 2.

5.2 Maximum achievable throughput

As presented in Chapter 4.2.2, this section evaluates the maximum achievable throughput between two DECT NR+ nodes implemented using Nordic's nRF9161 development kits. A

comparison between theoretical and measured throughput values is discussed for each MCS level, highlighting performance limitations introduced by firmware constraints and system behavior. Furthermore, no LBT or HARQ mechanisms were used in these test cases, in order to evaluate the maximum achievable PHY-layer throughput.

5.2.1 Throughput performance using Nordic's DeSh

As discussed earlier in Chapter 4.2.2, DeSh firmware includes a built-in scheduling delay between consecutive transmissions, controlled via the `-c_gap_subslots` parameter. This parameter introduces intentional gaps between packets, limiting the maximum achievable throughput. Using the default `-c_gap_subslots = 0` setting and the highest available MCS and sub-slot configuration, the maximum throughput approximately 2240 kbit/s was observed. However, current waveform analysis using the PPK2 revealed that this configuration introduces a transmission gap of approximately 830 μs between consecutive packets. By adjusting the `-c_gap_subslots` parameter to 3, the TX gap was reduced to approximately 630 μs , resulting in an improved throughput of approximately 2450 kbit/s, as illustrated in Figure 5.10. Attempts to reduce this delay further (e.g., setting the parameter to 1 or 2) resulted in modem scheduling errors, likely due to modem handling delays.

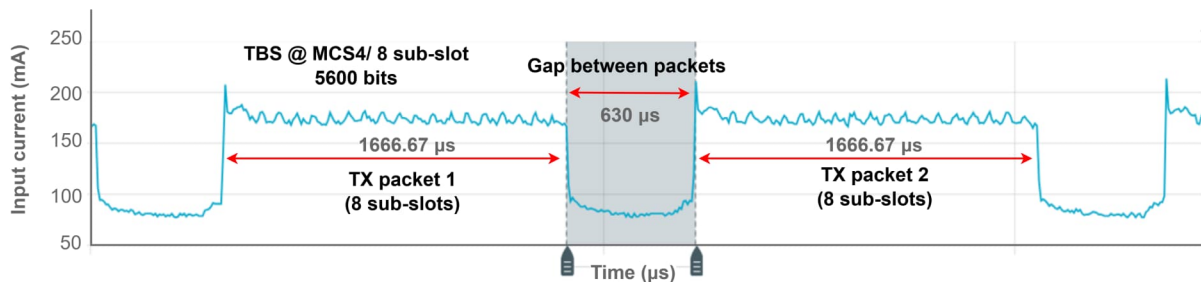


Figure 5.10. Illustrative current consumption profile of a DECT NR+ transmitter with the scheduling gap between two neighboring packets highlighted for DeSh.

5.2.2 Throughput performance using custom firmware

Using the Table 3.2 in Chapter 3.3.3, theoretical maximum throughput can be calculated from the TBS for each MCS and sub-slot index, using the following expression:

$$\text{Max Theoretical Throughput (kbit/s)} = \frac{\text{TBS (bits)}}{\text{Time per sub-slot (ms)} \times \text{Number of sub-slots per second}} \quad (5.6)$$

Given that a DECT NR+ frame is 10 ms in duration and contains 48 sub-slots, the time per sub-slot is known. The TBS values, which represent payload capacity (excluding PHY and control overhead), are used to compute the theoretical maximum throughput for each MCS and sub-slot combination. The custom firmware developed for this study was used to evaluate practical throughput under minimal delay conditions, without any firmware-imposed TX gaps, except modem latencies. The current waveform, shown in Figure 5.11, indicates that the minimum achievable TX gap is approximately 570 μs , which closely aligns with the sum of the modem's internal transition delays:

- Idle to TX : 430 μs
- TX to Idle : 100 μs

giving a total of approximately 530 μs of gap between TX sessions. These reference latency values were obtained using the `nrf_modem_dect_phy_capability_get()` API. Additional modem timing characteristics are discussed in Chapter 5.3.

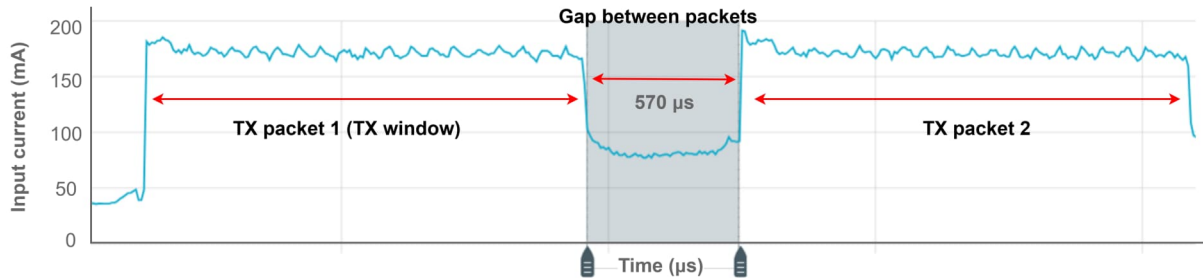


Figure 5.11. Illustrative current consumption profile of a DECT NR+ transmitter with the scheduling gap between two neighboring packets highlighted for custom firmware.

The Table 5.6 presents both the measured and theoretically calculated throughput values for the maximum number of sub-slots supported at each MCS level. The achieved throughput is expressed as the ratio of measured throughput to theoretical throughput.

Table 5.6. Theoretical and measured throughput for each MCS level.

MCS	Sub-slots	TBS (bits)	Theoretical throughput (kbit/s)	Measured throughput (kbit/s)	Achieved throughput (%)
0	16	1992	597.60	518.11	87%
1	16	3960	1188.00	1020.94	86%
2	15	5536	1771.52	1496.98	85%
3	11	5280	2304.00	1845.70	80%
4	8	5600	3360.00	2498.70	74%

Based on the Table 5.6 data, the MCS levels and their corresponding maximum throughput values (both measured and theoretical) are illustrated in Figure 5.12. Furthermore, throughput measurements were conducted across all MCS/sub-slot combinations, to fully characterize system behavior under varying PHY configurations and presented in Table 5.7.

Table 5.7. Measured throughput (kbit/s) for each MCS and sub-slot combination.

MCS\sub-slots	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0.00	151.75	238.49	303.72	350.55	381.79	406.34	425.78	448.70	461.85	472.66	482.21	490.57	497.62	512.78	518.11
1	45.92	329.09	496.37	622.18	712.31	772.12	819.12	856.23	880.64	907.33	929.30	948.66	965.61	979.94	1010.13	1020.94
2	80.27	505.44	766.12	943.12	1059.40	1148.29	1194.29	1251.70	1325.65	1363.85	1395.23	1415.38	1459.26	1478.93	1496.98	-
3	126.05	680.66	1005.42	1248.81	1392.31	1510.73	1603.77	1676.93	1754.33	1804.44	1845.70	-	-	-	-	-
4	206.06	1027.91	1533.04	1838.91	2066.18	2239.04	2393.99	2498.70	-	-	-	-	-	-	-	-

According to the results presented in Figure 5.13, DECT NR+, as implemented on the nRF9161 SiP, is capable of achieving a maximum throughput of approximately 2500 kbit/s using MCS 4 with the maximum supported allocation of 8 sub-slots. This performance is primarily constrained by the modem's hardware limitations and the currently supported MCS and sub-slot configurations. The nonlinear trend observed in the measured throughput values (Figure 5.13) can be further explained through the theoretical analysis shown in Figure 5.14, which plots throughput values calculated using Equation 5.6. While both the TBS and the total

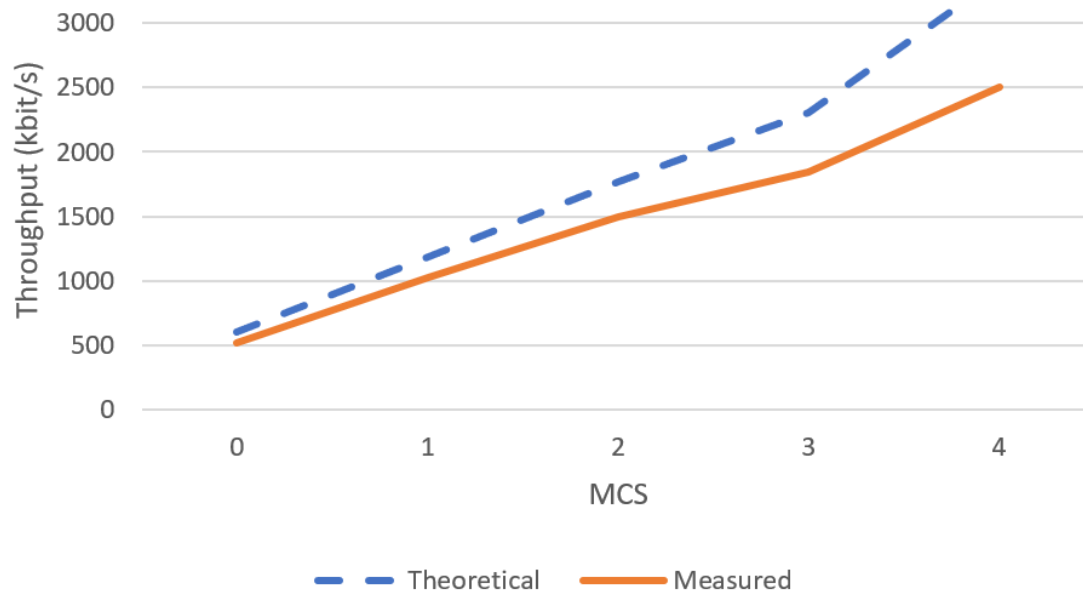


Figure 5.12. Comparison of maximum theoretical and measured throughput for different MCS levels.

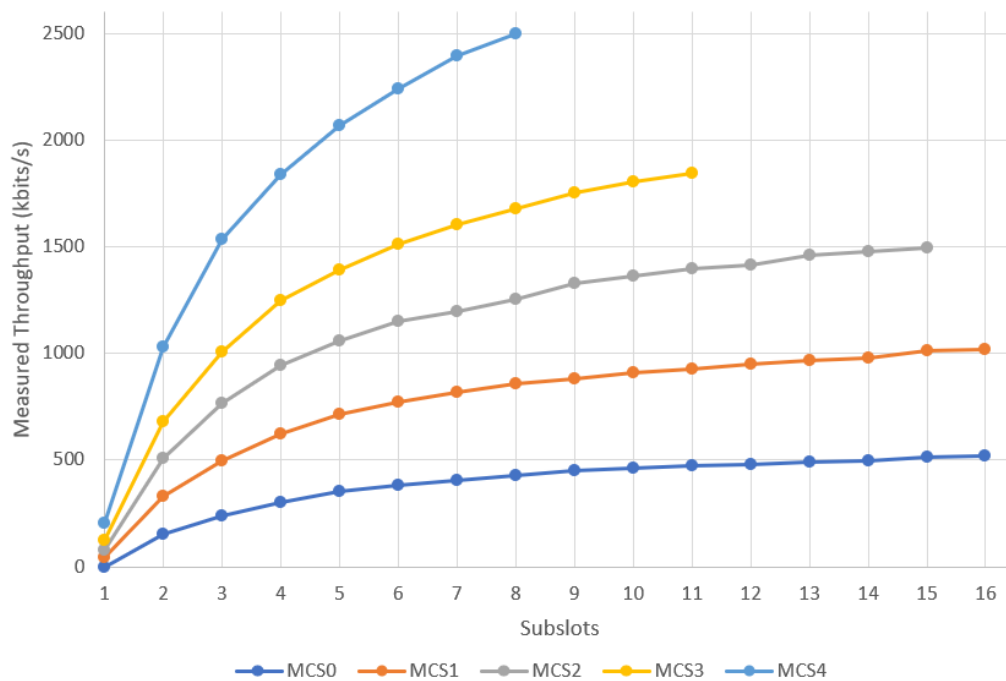


Figure 5.13. Measured throughput (kbit/s) for all available MCS/sub-slot combinations on nRF9161.

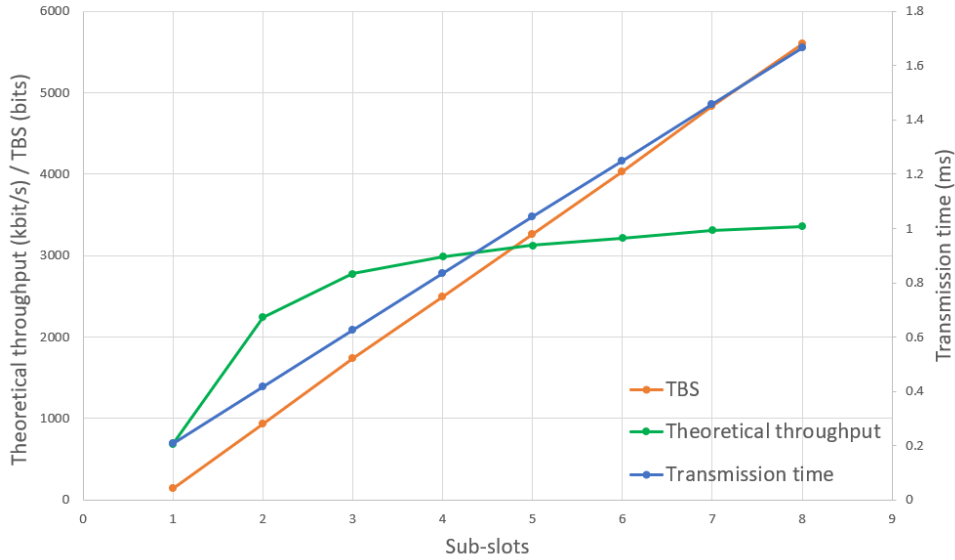


Figure 5.14. Variation of TBS, throughput, and total transmission time for MCS 4 as a function of the number of occupied sub-slots.

TX time (i.e., the product of sub-slot duration and sub-slots per second) increase almost linearly, they do so at different rates. Since throughput is defined as the ratio of TBS to total TX time, this mismatch in growth rates results in the nonlinearity of the throughput curve. Specifically, in lower sub-slot configurations, the TBS increases more rapidly than the transmission time, resulting in a sharp rise in throughput. However, as the number of sub-slots increases, the growth in TBS becomes more gradual relative to the growth in time, leading to a plateauing effect. This behavior gives rise to the concave shape of the throughput curve, despite both parameters appearing to increase linearly. This behavior can be better understood by referring to the MCS configurations summarized in Table 3.1. In DECT NR+, transitions from MCS 1 to 2 and from MCS 3 to 4 involve changes in coding rate only, without any increase in modulation order. As a result, the TBS growth between those MCS levels is limited. Understanding this nonlinear relationship is particularly important when optimizing PHY configurations, as it highlights that allocating additional sub-slots does not always yield proportional throughput gains, especially at higher sub-slot counts.

According to the literature, BLE 5.0 achieves a maximum theoretical throughput of approximately 1.4 Mbit/s [14]. In comparison, this study demonstrates that DECT NR+ significantly outperforms BLE in terms of maximum achievable throughput, reaching up to 2.5 Mbit/s under optimal conditions with the current nRF9161 implementation. While these results already surpass some values reported in prior studies, such as the 2246.4 kbit/s [2] throughput achieved in one implementation, as described in Chapter 2.2, they remain below certain analytical estimates found in literature. For example, a theoretical upper-bound throughput of 12.47 Mbit/s [5] was predicted under ideal conditions, assuming the use of higher-order modulation schemes like 1024-QAM, as discussed in Chapter 2.5. In future DECT NR+ implementations, support for additional MCS levels and higher-order modulations such as 1024-QAM as outlined in the ETSI specification [6] could enable significantly higher data rates. Nevertheless, the measured performance is already satisfactory given DECT NR+ is not primarily intended for high-throughput applications. Instead, the technology is optimized for low-latency, high-reliability communication in dense device environments. However, unlocking higher modulation levels in future hardware or firmware updates could allow DECT NR+ to evolve into a more versatile and

well-balanced communication solution.

5.3 Minimum latency measurement

As discussed in Chapter 4.2.3, a custom firmware was primarily used for RTT measurements, since the values obtained using the DeSh ping command were significantly higher than the expected performance of DECT NR+. For the measurements presented in this section, the default parameters from the `hello_dect_phy` sample application were used. TX power +19 dBm and MCS 1. Additionally, LBT and HARQ was disabled to minimize TX startup delay and ensure consistent timing. A subset of RTT measurement samples, extracted from all recorded TX trials, is presented in Table 5.8. In total, over 1000 RTT samples were collected and used for the analysis.

Table 5.8. Sample RTT values in modem ticks and milliseconds.

Test no.	Modem ticks (t2 - t1)	RTT (ms)
1	156185	2.260
2	156430	2.263
3	156232	2.260
4	156043	2.258
5	156371	2.262

Based on the collected data, the minimum, maximum, mean, and standard deviation of RTT were computed using MATLAB. The distribution of RTT values is illustrated in the histogram shown in Figure 5.15.

- Minimum RTT: 2.255 ms
- Maximum RTT: 2.280 ms
- Mean RTT: 2.264 ms
- Standard Deviation: 0.006 ms

As shown in the flowchart in Figure 4.7, the firmware imposed delays contributing to RTT measurements primarily include:

- On the server side: the time required for packet reception condition checking and PHY header processing
- On the client side: the time taken to request the modem timestamp (t1) via the API and to wait for the corresponding callback, followed by the capture of the modem timestamp t2 and RTT calculation inside the `pdc()` callback.

These delays can be considered negligible in practice, as the nRF9161 SiP operates with a 64 MHz system clock, enabling fast execution of low-level operations. While a single clock cycle corresponds to approximately 16 ns, the actual processing of headers and API-level functions typically requires several cycles. However, even accounting for the overhead introduced by the Zephyr RTOS and firmware abstraction layers, it is reasonable to assume that such operations are completed within a few microseconds. Consequently, their contribution to overall latency is minimal compared to modem transition delays. In the context of RTT measurements, the dominant contribution to latency arises from modem state transitions, particularly between RX and TX modes. The relevant modem latency values were obtained using the `nrf_modem_dect_phy_capability_get()` API and are listed in Table 5.9.

Using these latency values, the ideal RTT timeline based on the flow shown in Figure 4.7 is illustrated in Figure 5.16. The air time (TX duration) was verified using current waveform measurements from the PPK2 and was found to match the expected value for a 12 byte RTT

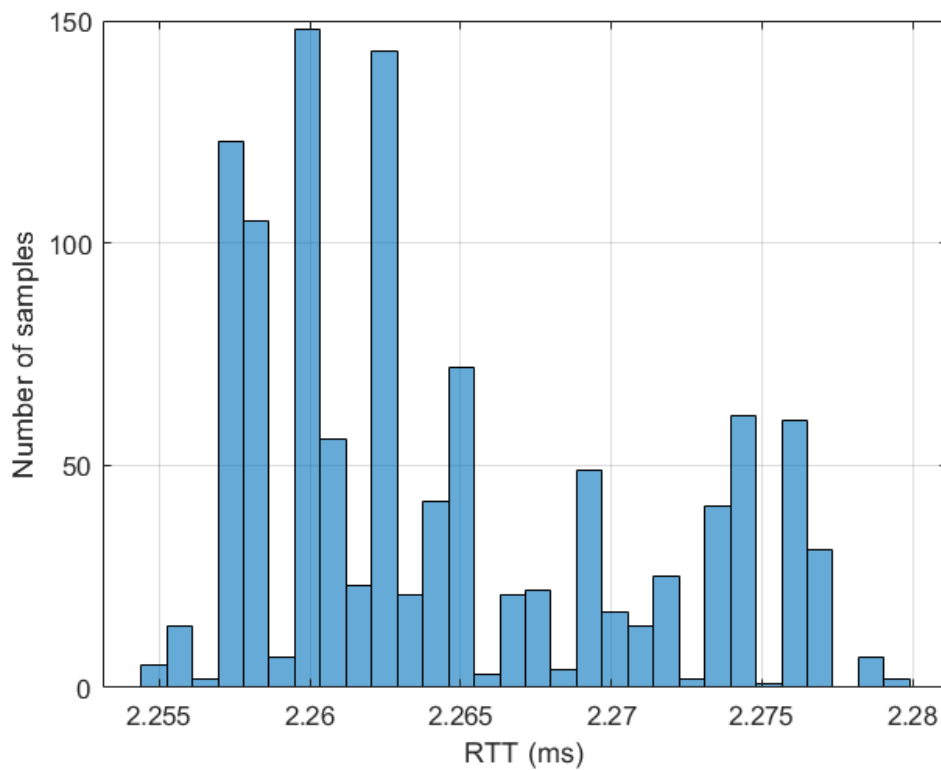


Figure 5.15. Distribution of measured RTT values.

Table 5.9. nRF9161 modem latencies.

Operation	Modem ticks (69120 kHz)	Time (μ s)
from Idle to RSSI RF sampling start	22118	320
from RF sampling of RSSI to Idle	15206	220
from Idle to RX RF sampling start	22118	320
from RF sampling of RX to Idle	10713	155
from RF sampling of RX with RSSI measurement to Idle	13478	195
from RX stop request to RF sampling stop	14169	205
from Idle to TX RF sample transmission start	29721	430
from Idle to TX with LBT monitoring RF sampling start	37324	540
from TX RF sample transmission end (with or without LBT) to Idle	6912	100
from Idle to TX with LBT monitoring RF sampling start in TX-RX operation	39744	575
from TX RF sample transmission end to RX RF sampling start in TX-RX operation	20044	290

packet. In this analysis, propagation delays are excluded, as they are negligible compared to other latency components. Based on this timing analysis, the minimum theoretical RTT between two DECT NR+ nodes implemented on nRF9161 development kits configured with MCS 1 and 2 sub-slots, which support up to 37 bytes per RTT packet, can be expressed as:

$$RTT_{ideal} = 1897 \mu s \quad (5.7)$$

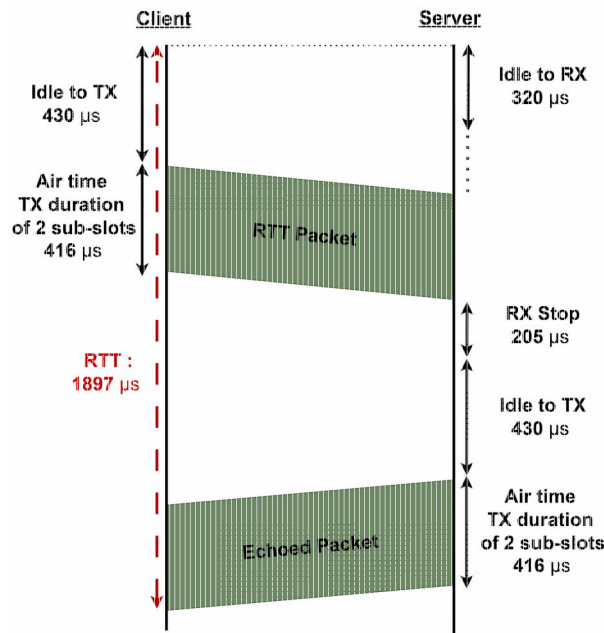


Figure 5.16. Expected RTT components and modem delays.

This ideal RTT value is comparable with the measured mean RTT of $2264 \mu s$, which includes additional firmware processing and OS level data handling delays within Zephyr. Given this, it can be concluded that DECT NR+ is capable of achieving sub-millisecond one-way latency under ideal conditions. The measured RTT indicates a minimum one-way latency of approximately 1 ms, demonstrating the protocol's suitability for low-latency communication applications.

Further comparison with values reported in the literature supports this conclusion. In a study evaluating DECT NR+ for professional audio applications, a minimum PHY level user-plane latency of 1.062 ms was achieved at MCS 1, corresponding to an RTT of approximately 2.122 ms [2]. Another study reported a one-way latency of 0.9776 ms (RTT of 1.9552 ms) with HARQ disabled and using a 2 sub-slot TX duration [3]. Additionally, an ETSI TR states that the DECT-2020 NR standard is capable of achieving a user-plane one-way latency of 0.9 ms at MCS 1, resulting in an RTT of 1.8 ms [4]. Considering all these references alongside the ideal RTT of $1897 \mu s$ derived in this study for a 2 sub-slot TX duration with a maximum payload of 37 bytes, it is reasonable to conclude that DECT NR+ operates at the threshold of sub-millisecond one-way latency under optimal conditions. However, its ability to maintain such performance may diminish with significantly larger payloads. In comparison, as reported by an industry website, BLE typically exhibits latency values around 6 ms [15]. These results clearly highlight the superior latency performance of DECT NR+, validating its potential for time-critical applications requiring fast and reliable data exchange.

5.4 Energy efficiency estimation

In this section, the energy efficiency of DECT NR+ transmissions is evaluated based on current waveform measurements captured using the PPK2. Measurements were taken for various MCS and sub-slot combinations, and further analysis was performed to calculate energy per bit during TX. Additional power consumption measurements were also conducted for RX mode, as well as during modem initialization and de-initialization states.

5.4.1 TX energy per bit analysis

In this test scenario, the current and duration of a single packet TX were recorded. For each configuration, the TX duration was determined from the high-current window as shown in Figures 5.10 and 5.11. Energy per bit was calculated across different MCS and sub-slot combinations. Results for MCS 0 are presented in Table 5.10.

Table 5.10. TX energy consumption for MCS 0.

Sub-slots	TBS (bytes)	TX duration (μ s)	Average current (mA)	Power consumption (W)	Energy per bit (μ J)
2	17	380	185.65	0.928	2.5936
3	33	590	185.92	0.930	2.0775
4	50	800	186.82	0.934	1.8682
5	67	1000	185.57	0.928	1.7311
6	83	1220	186.17	0.931	1.7103
7	99	1420	187.35	0.937	1.6795
8	115	1630	185.97	0.930	1.6475
9	133	1830	187.08	0.935	1.6088
10	149	2060	185.56	0.928	1.6034
11	165	2300	185.87	0.929	1.6193
12	181	2510	186.58	0.933	1.6171
13	197	2710	186.26	0.931	1.6014
14	213	2920	186.00	0.930	1.5937
15	233	3120	186.13	0.931	1.5577
16	249	3340	186.02	0.930	1.5595

Configurations with sub-slot = 1 are omitted from the analysis for a better visualization, particularly for MCS 0, where the TBS is 0 bytes (as shown in Table 3.2). Additionally, the sub-slot count shown corresponds to the actual TX sub-slots, which is one greater than the sub-slot index. Comprehensive measurements were conducted across all MCS and sub-slot combinations using a fixed TX power of +19 dBm. The resulting energy per bit values are illustrated in Figure 5.17.

As shown in the Figure 5.17, the most energy-efficient configuration is achieved using the highest available MCS, where the minimum energy per bit reaches approximately $0.26 \mu\text{J/bit}$. In contrast, MCS 0 results in the highest energy cost, exceeding $1.5 \mu\text{J/bit}$, with a maximum value of $2.6 \mu\text{J/bit}$ which is ten times higher than the minimum. This indicates that using the highest MCS offers the best energy efficiency, but it reduces reliability and communication range. Therefore, such configurations are most suitable for dense network environments, where devices are placed in close proximity. Based on a technical comparison report [16], BLE achieves an energy consumption of approximately $0.075 \mu\text{J/bit}$, which is significantly lower than the minimum value observed in this study for DECT NR+, measured at $0.26 \mu\text{J/bit}$. This indicates that BLE outperforms DECT NR+ in terms of energy efficiency. However, this comes at the cost of reduced communication range and lower achievable throughput compared to DECT NR+.

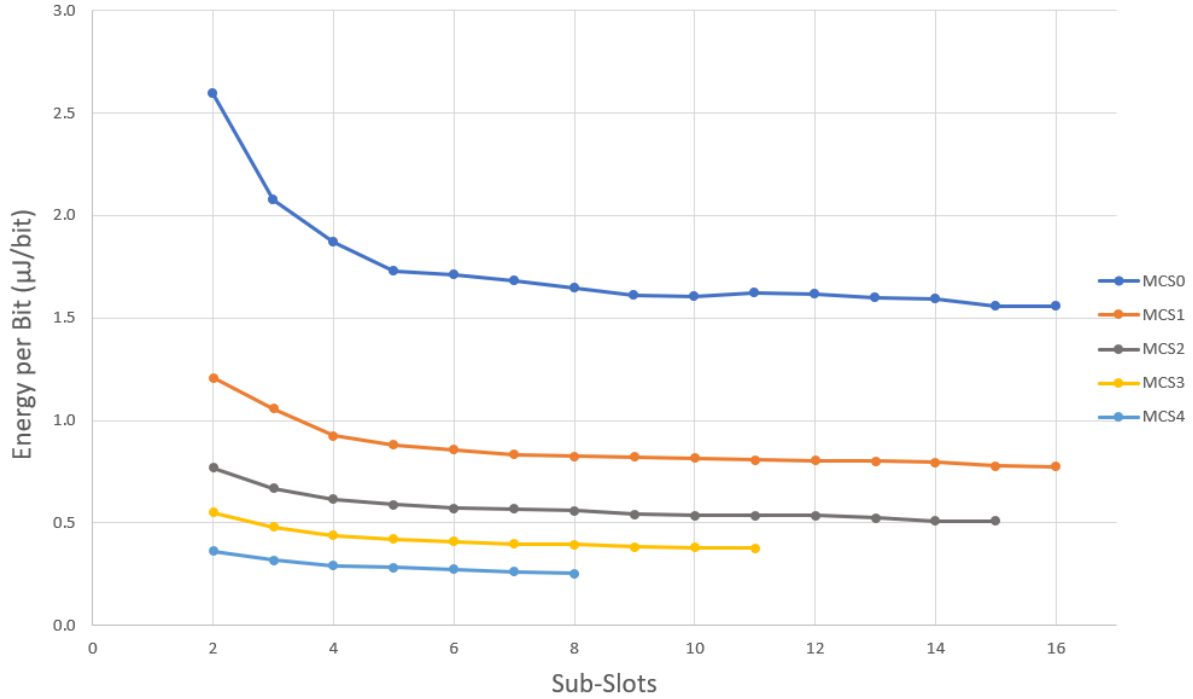


Figure 5.17. Energy consumption in TX under different MCS/sub-slots.

5.4.2 Impact of TX power on energy consumption

To evaluate TX power characteristics, the peak and average current consumption were measured across all available TX power levels. Measurements were taken using the PPK2 at MCS 1 and MCS 4, with each test based on a 7 sub-slot packet duration. Table 5.11 presents the peak current, average current, and the corresponding PAPR, calculated using the following expression:

$$\text{PAPR (\%)} = \frac{I_{\text{peak}} \text{ (mA)}}{I_{\text{avg}} \text{ (mA)}} \times 100\% \quad (5.8)$$

This calculation assumes a constant supply voltage of 5 V throughout the experiments.

Table 5.11. Current consumption and PAPR for MCS 1 and MCS 4.

MCS 1				MCS 4			
TX power (dBm)	I peak (mA)	I avg (mA)	PAPR	TX power (dBm)	I peak (mA)	I avg (mA)	PAPR
-12	94.88	77.99	122%	-12	94.88	78.60	121%
-8	97.87	79.63	123%	-8	94.88	79.30	120%
-4	95.63	79.67	120%	-4	97.12	80.11	121%
0	94.14	83.61	113%	0	99.36	84.31	118%
4	96.38	87.90	110%	4	104.59	87.84	119%
7	98.62	91.42	108%	7	109.08	91.91	119%
10	109.83	101.55	108%	10	116.57	102.02	114%
13	124.82	115.38	108%	13	129.33	116.27	111%
16	148.91	139.57	107%	16	152.68	140.56	109%
19	192.87	182.72	106%	19	200.48	184.28	109%
21	191.34	182.60	105%	21	196.67	183.68	107%
23	195.15	183.29	106%	23	197.43	183.57	108%

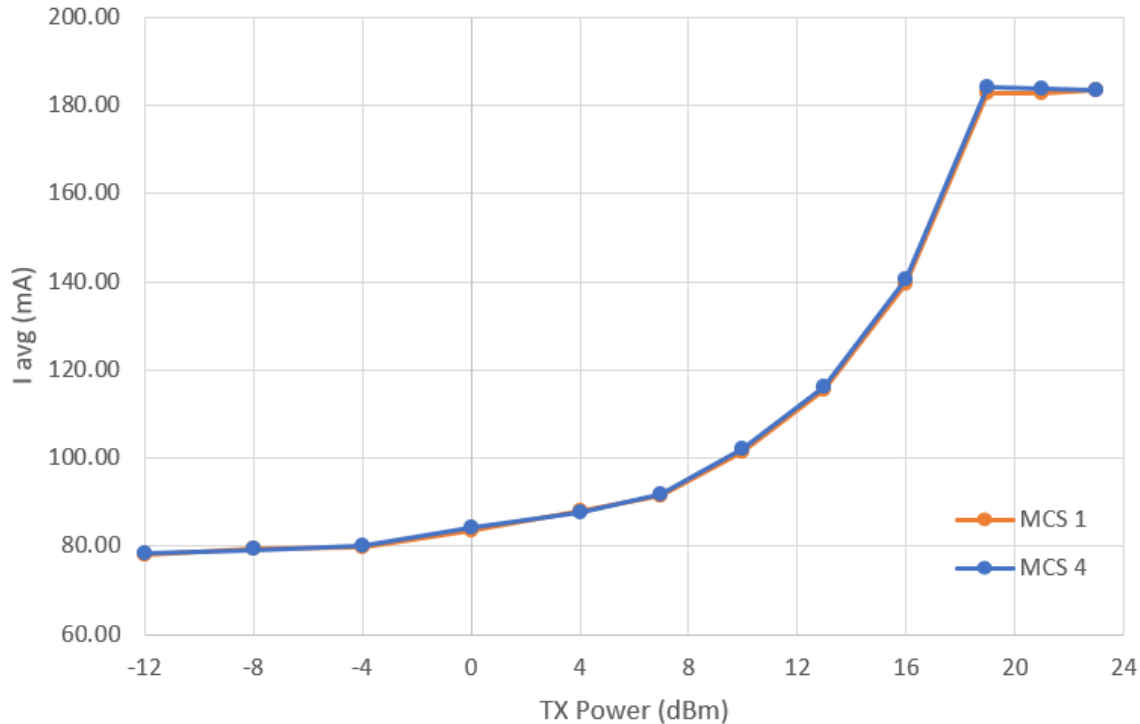


Figure 5.18. Average TX current for MCS 1 and MCS 4.

According to the Figure 5.18, although the DECT-2020 NR standard defines a maximum TX power of +23 dBm for transceivers, the nRF9161 supports only up to +19 dBm, as stated in its technical documentation [17]. This is confirmed by the absence of any noticeable current increase beyond the +19 dBm level. While higher power values are accepted by the modem, the hardware imposes a limit at +19 dBm. It is also noteworthy that the overall average current consumption does not significantly vary with the selected MCS level, as both traces align closely. At the minimum TX power of -12 dBm, the average current consumption during TX is approximately 78 mA, corresponding to an average power of around 390 mW. This is considerably higher compared to BLE, which consumes only 16.4 mA at +8 dBm, resulting in an average power consumption of approximately 50 mW [18]. This PAPR metric is not related to the RF waveform, but rather reflects system-level power efficiency from a hardware perspective. Both MCS 1 and MCS 4 converge to a PAPR of approximately 108% at maximum TX power.

5.4.3 RX mode power consumption

This test focused on measuring receiver power consumption under different modem states:

- While receiving a packet
- While waiting to receive
- When modem initialized (with no active RF operations)
- When modem de-initialized (processor only)

Deep sleep modes were not configured for this study. Power management was handled by Zephyr's default scheduler, based on the thread activity and state transitions. As shown in Table 5.12, the modem consumes significantly more power than the main processor, even when no RF operations are active. The MCS level used during reception has minimal impact on RX

Table 5.12. Measured current and power consumption across operating modes.

Mode	MCS	RSSI (dBm)	SNR (dB)	Avg current (mA)	Avg power (mW)	Notes
RX Active	1	-32.00	25.50	38.86	194.30	RF reception in progress
RX Idle	1	-32.00	25.50	41.88	209.40	Idle in RX mode, awaiting packet
RX Active	4	-29.50	25.00	40.79	203.95	RF reception in progress
RX Idle	4	-29.50	25.00	41.66	208.30	Idle in RX mode, awaiting packet
Modem Initialized	-	-	-	30.93	154.65	Modem initialized, no RF activity
Modem De-initialized	-	-	-	2.24	11.20	Firmware main loop idle
Modem De-initialized	-	-	-	0.986	4.93	System idle with no active threads

power consumption. To reduce power usage in idle or low duty-cycle applications, disabling the modem or enabling it only when necessary can lead to substantial energy savings. It is also noteworthy that on the nRF9161 SiP, the average power consumption during RX idle is slightly higher than during active reception for both MCS cases. This suggests that when the modem is passively listening for a signal, without being locked to an incoming packet, the RF front end may consume more power in its effort to detect a signal. In contrast, when the RX path is active and synchronized to a signal, the RF front end power usage appears to decrease, despite continuous digital processing.

When comparing the average power consumption between TX and RX modes, at the lowest transmit power level of -12 dBm, the TX mode draws approximately 390 mW, while RX mode consumes around 200 mW, roughly half the TX power. However, in low duty-cycle systems with continuous reception mode, RX power consumption still plays a significant role in the overall energy budget. For comparison, BLE RX mode typically consumes around 6.26 mA [18], resulting in an average power of approximately 19 mW, which is substantially lower than that of DECT NR+.

6 CONCLUSION AND FUTURE WORK

This chapter summarizes the key findings of the research, evaluates how the objectives were met, and outlines potential directions for future work. The primary focus of this thesis was the real-world performance evaluation of DECT NR+ at the PHY layer using Nordic Semiconductor's nRF9161 development kits.

The experimental work focused on four main objectives:

- Maximum communication range of a single hop
- Maximum achievable throughput
- Minimum latency (based on RTT)
- Energy efficiency estimation

6.1 Summary of key findings

For the maximum communication range evaluation, Nordic's sample application DeSh was used. The key performance indicator was the PDR, the ratio of successfully received packets to transmitted packets. Tests were conducted in both indoor and outdoor environments. In addition to PDR, variations in RSSI and SNR were also observed.

- In outdoor LOS conditions, the maximum range achieved was 730 meters, maintaining approximately 80% PDR at a TX power of +19 dBm using MCS 0.
- Under less favorable outdoor conditions, environmental factors such as interference and partial obstructions reduced the reliable range to approximately 400 meters, despite using the same configuration.
- Indoors, the maximum range achieved was 49 meters with 80% PDR using +4 dBm TX power and MCS 4.

For the maximum achievable throughput, the firmware was optimized to eliminate software-included delays and reveal the upper limits constrained by the modem itself. By analyzing high-current bursts during TX, it was determined that the minimum scheduling gap between transmissions was approximately $570 \mu s$. Under these conditions, using MCS 4 and transmitting for 8 sub-slots, a peak throughput of 2498.70 kbit/s was achieved. This value reflects the data channel throughput only, excluding PHY layer overheads. A comprehensive set of throughput measurements for all supported MCS/sub-slot combinations was also presented. In the latency analysis, RTT measurements were performed using a custom implementation based on modem time-stamping. The minimum RTT recorded was 2.255 ms, implying a one-way PHY-layer latency of 1.127 ms under MCS 1, +19 dBm TX power, LBT disabled, and a TX duration of 2 sub-slots. Additionally, an ideal one-way latency estimation using modem-reported delays was found to be approximately $950 \mu s$.

For energy efficiency, energy per bit was calculated across various MCS and sub-slot configurations. Further analysis was conducted on TX mode power behavior and system-level PAPR using Nordic's PPK2. The results show that the minimum energy consumption achieved was $0.26 \mu J/bit$ at MCS 4, while the average RX power consumption was approximately 200 mW.

In summary, the experimental results demonstrated that DECT NR+ is capable of achieving:

- A maximum range of 730 meters with 80% PDR at a throughput of 368.05 kbit/s and 655 meters with 100% PDR.
- A maximum throughput of 2498.70 kbit/s.
- A minimum RTT of 2.255 ms.
- A minimum energy consumption of $0.26 \mu J/bit$.
- A minimum average TX power of 390 mW and average RX power of 200 mW.

These results highlight that DECT NR+ is a strong potential candidate for URLLC oriented applications, particularly in time-sensitive wireless communication environments. Compared to BLE, DECT NR+ demonstrated superior performance in communication range, peak throughput, and latency. While BLE typically achieves LOS ranges of 200-400 meters and a maximum theoretical throughput of 1.4 Mbit/s, DECT NR+ maintained communication up to 730 meters with 80% PDR and achieved up to 2.5 Mbit/s throughput. However, BLE remains more energy-efficient, consuming as little as $0.075 \mu J/bit$ compared to the best-case $0.26 \mu J/bit$ in this study. This highlights a trade-off between energy consumption and range/throughput, where DECT NR+ is more suitable for long-range, high-throughput, and low-latency use cases.

The experimental results are also in line with several prior studies and theoretical expectations. For example, previous work reported RTT values of around 1.95-2.12 ms, comparable to the 2.264 ms mean RTT measured in this study. Similarly, reported maximum throughput values near 2.2 Mbit/s and observed RSSI, SNR thresholds for 100% PDR match the trends captured here. However, other studies have demonstrated multi-kilometer range using optimized antennas and elevated deployments. The modest range observed in this study, limited to 730 meters, is attributed to hardware constraints and environmental conditions, such as low antenna height and urban interference. Despite this, the findings validate DECT NR+ as a reliable URLLC-capable PHY solution in practical embedded applications.

6.2 Limitations

This research was limited to point-to-point PHY-layer evaluations using Nordic's currently available modem firmware. Features such as MAC-layer functionality, multi-node communications, interference handling, mesh networking, and HARQ-based retransmission strategies were not explored. All measurements were performed using the built-in antennas on the nRF9161 development kits.

6.3 Future work

Future studies can expand upon this work by:

- Evaluating MAC-layer performance and scheduling algorithms.
- Evaluating performance under high-density deployments and interference-prone environments.
- Implementing and testing multi-hop mesh topologies to study scalability and latency trends.
- Exploring how HARQ mechanisms influence range, throughput, and latency.
- Assessing the impact of using external antennas, supported via the development kit's external antenna port, on communication performance.

This thesis contributes one of the first experimental evaluations of DECT NR+, establishing a foundational understanding of its PHY-layer capabilities. As wireless systems continue to evolve, DECT NR+ shows promise as a flexible and efficient solution for future URLLC and mMTC applications.

7 REFERENCES

- [1] M. M. Haque, J. S  e, J. Pirskanen, and M. Valkama, “Dect-2020 nr link distance performance in varying environments: Models and measurements,” in *2025 19th European Conference on Antennas and Propagation (EuCAP)*, pp. 1–5, 2025.
- [2] J. D  rre and N. Werner, “Dect-2020 nr for professional live audio: Design space exploration and practical evaluation,” in *2024 IEEE 5th International Symposium on the Internet of Sounds (IS2)*, pp. 1–7, 2024.
- [3] V. Dhanwani, N. Kumar, A. K. Bachkaniwala, D. Rawal, and S. Kumar, “Assessment of candidate technology etsi: Dect-2020 new radio,” in *2020 IEEE 3rd 5G World Forum (5GWF)*, pp. 625–630, 2020.
- [4] ETSI Technical Committee DECT, “System Reference Document (SRdoc): DECT-2020 NR technology operating in frequency bands below 6 GHz,” Technical Report TR 103 943 V1.1.1, ETSI, January 2024. Available: https://www.etsi.org/deliver/etsi_tr/103900_103999/103943/01.01.01_60/tr_103943v010101p.pdf.
- [5] R. C. Kizilirmak and S. Askarov, “Examination of phy layer features in dect-2020 and lte-m networks,” in *2024 IEEE 10th Information Technology International Seminar (ITIS)*, pp. 200–204, 2024.
- [6] ETSI, “DECT-2020 New Radio (NR); Part 1: Overview; Release 2,” Tech. Rep. TS 103 636-1 V2.1.1, ETSI, 2024. https://www.etsi.org/deliver/etsi_ts/103600_103699/10363601/02.01.01_60/ts_10363601v020101p.pdf (4.6.2025).
- [7] ETSI, “DECT-2020 New Radio (NR); Part 4: MAC layer; Release 1,” Tech. Rep. TS 103 636-4 V1.5.1, ETSI, 2024. https://www.etsi.org/deliver/etsi_ts/103600_103699/10363604/01.05.01_60/ts_10363604v010501p.pdf (Accessed: 4.6.2025).
- [8] ETSI, “DECT-2020 New Radio (NR); Part 5: DLC and Convergence layers; Release 2,” Tech. Rep. TS 103 636-5 V2.1.1, ETSI, 2024. https://www.etsi.org/deliver/etsi_ts/103600_103699/10363605/02.01.01_60/ts_10363605v020101p.pdf (Accessed: 4.6.2025).
- [9] ETSI, “DECT-2020 New Radio (NR); Part 3: Physical layer; Release 1,” Tech. Rep. TS 103 636-3 V1.5.1, ETSI, 2024. https://www.etsi.org/deliver/etsi_ts/103600_103699/10363603/01.05.01_60/ts_10363603v010501p.pdf (Accessed: 4.6.2025).
- [10] Nordic Semiconductor, *DECT PHY API — nRF Modem library*. Nordic Semiconductor ASA, 2025. https://docs.nordicsemi.com/bundle/ncs-latest/page/nrfxlib/nrf_modem/doc/dectphy.html (Accessed: 12 June 2025).
- [11] ETSI, “DECT-2020 New Radio (NR); Part 2: Radio reception and transmission requirements; Release 1,” Tech. Rep. TS 103 636-2 V1.5.1, ETSI, 2024. https://www.etsi.org/deliver/etsi_ts/103600_103699/10363602/01.05.01_60/ts_10363602v010501p.pdf (Accessed: 4.6.2025).
- [12] Nordic Semiconductor, *Modem library (nRF Modem) — nrfxlib README*. Nordic Semiconductor ASA, 2025. https://docs.nordicsemi.com/bundle/ncs-latest/page/nrfxlib/nrf_modem/README.html (Accessed: 8.6.2025).
- [13] SECUR Project Consortium, “Technical document report: Bluetooth low energy 5.0,” tech. rep., European Commission H2020 Project SECUR, WP2, 2021. <https://www.secur-project.eu> (Accessed: 4.6.2025).
- [14] Z. Zhang, M. Siekkinen, Y. Xiao, and A. Yl  -J   ski, “Demystifying the characteristics of bluetooth low energy (ble) traffic on smartphones,” *ACM Transactions on Sensor Networks (TOSN)*, vol. 15, no. 3, pp. 1–31, 2019.

- [15] MOKOSmart, “Bluetooth vs bluetooth low energy: A detailed comparison.” <https://www.mokosmart.com/bluetooth-vs-bluetooth-low-energy-a-detailed-comparison/>, 2024. (Accessed: 9 June 2025).
- [16] Digi-Key Electronics, “Comparing low-power wireless technologies (part 2),” tech. rep., Digi-Key Electronics, 2017. <https://www.digikey.com/en/articles/comparing-low-power-wireless-technologies-part-2> (Accessed: 9June2025).
- [17] Nordic Semiconductor, *nRF9161 Product Specification v1.0*. Nordic Semiconductor ASA, 2023. https://www.mouser.fi/datasheet/2/297/nRF9161_PS_v1_0-3439621.pdf (Accessed: 4.6.2025).
- [18] Nordic Semiconductor, *nRF52840 SoC Product Brief Version 3.0*. Nordic Semiconductor ASA, 2024. <https://www.nordicsemi.com/nRF52840> (Accessed: 4.6.2025).

8 APPENDICES

Appendix 1 DeSh modified section on "startup_cmd_ctrl.c"

```

static void startup_cmd_worker(struct k_work *work_item)
{
    int len = 0;
    int err;

    struct k_work_delayable *delayable_work =
        k_work_delayable_from_work(work_item);
    struct startup_cmd_worker_data *data_ptr =
        CONTAINER_OF(delayable_work, struct
            startup_cmd_worker_data, work);
    struct startup_cmd current_cmd_data;

    /* Read the startup command string from the given memory slot
       */
    err = startup_cmd_settings_cmd_data_read_by_memslot(
        &current_cmd_data, data_ptr->running_mem_slot);
    if (err) {
        desh_error("(%)s):_Cannot_read_startup_cmd_data_for_
            mem_slot_%d",
            __func__, data_ptr->running_mem_slot);
        return;
    }

    /* Execute command if present */
    len = strlen(current_cmd_data.cmd_str);
    if (len) {
        desh_print("Starting_to_execute_sett_cmd_%d\"%s\"...
            ",
            data_ptr->running_mem_slot, current_cmd_data.
                cmd_str);
        shell_execute_cmd(desh_shell, current_cmd_data.
            cmd_str);
    }

    /* Move to the next memory slot */
    startup_cmd_worker_data.running_mem_slot++;
    if (startup_cmd_worker_data.running_mem_slot >
        STARTUP_CMD_MAX_COUNT) {
        startup_cmd_worker_data.running_mem_slot = 1; //
        Wrap around to first slot
    }

    /* Schedule next command execution with slot-specific delay
       */
    k_work_schedule_for_queue(
        &desh_common_work_q,
        &startup_cmd_worker_data.work,

```

```
        K_SECONDS(startup_cmd_settings_cmd_delay_get(  
            startup_cmd_worker_data.running_mem_slot)));  
    }
```

Appendix 2 RTT measurement program : Client

```
// Board A      RTT Measurement (Initiator)

#include <string.h>
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include <nrf_modem_dect_phy.h>
#include <modem/nrf_modem_lib.h>
#include <zephyr/drivers/hwinfo.h>
#include <zephyr/sys/printk.h>
#include <zephyr/sys/util.h>
#include <zephyr/device.h>
#include <zephyr/drivers/gpio.h>

LOG_MODULE_REGISTER(app);
BUILD_ASSERT(CONFIG_CARRIER, "Carrier_must_be_configured_properly.");

#define DATA_LEN_MAX 32
#define CONFIG_RX_PERIOD_S_mod 5

K_SEM_DEFINE(operation_sem, 0, 1);

static bool exit;
static uint16_t device_id;
static uint64_t latest_modem_time = 0;

struct __packed rtt_packet {
    uint32_t pkt_id;
    uint64_t tx_time_ticks;
};

static void init(const uint64_t *time, int16_t temp, enum
    nrf_modem_dect_phy_err err,
                const struct nrf_modem_dect_phy_modem_cfg *cfg)
{
    if (err) {
        LOG_ERR("Init_failed,_err_%d", err);
        exit = true;
        return;
    }
    k_sem_give(&operation_sem);
}

static void op_complete(const uint64_t *time, int16_t temperature,
    enum nrf_modem_dect_phy_err err, uint32_t
    handle)
{
    k_sem_give(&operation_sem);
}

static void deinit(const uint64_t *time, enum nrf_modem_dect_phy_err
```

```

    err)
{
    if (err) {
        LOG_ERR("Deinit failed, err %d", err);
        return;
    }
    k_sem_give(&operation_sem);
}

static void pcc(const uint64_t *time,
                const struct nrf_modem_dect_phy_rx_pcc_status *status
                ,
                const union nrf_modem_dect_phy_hdr *hdr)
{
    struct phy_ctrl_field_common *header = (struct
        phy_ctrl_field_common *)hdr->type_1;
    LOG_INF("Received header from device ID %d",
        header->transmitter_id_hi << 8 | header->
        transmitter_id_lo);
}

static void pcc_crc_err(const uint64_t *time,
                        const struct
                        nrf_modem_dect_phy_rx_pcc_crc_failure *
                        crc_failure)
{
    LOG_DBG("PCC CRC error at time %u" PRIu64, *time);
}

static void pdc(const uint64_t *time,
                const struct nrf_modem_dect_phy_rx_pdc_status *status
                ,
                const void *data, uint32_t len)
{
    const struct rtt_packet *rx_pkt = (const struct rtt_packet *)data
    ;

    uint64_t now = *time;
    uint64_t rtt_ticks = now - rx_pkt->tx_time_ticks;
    uint32_t rtt_ms = (uint32_t)(rtt_ticks / 69120);
    uint32_t rtt_us = (uint32_t)((rtt_ticks * 100 / 69120) % 100);

    printk("RTT: %u" PRIu64 " ticks, %lu.%02lu ms\n", rtt_ticks,
        rtt_ms, rtt_us);
}

static void pdc_crc_err(const uint64_t *time,
                        const struct
                        nrf_modem_dect_phy_rx_pdc_crc_failure *
                        crc_failure)
{
    LOG_DBG("PDC CRC error at time %u" PRIu64, *time);
}

```

```

}

static void rssi(const uint64_t *time, const struct
    nrf_modem_dect_phy_rssi_meas *status)
{
    LOG_DBG("RSSI_measurement_on_carrier_%d", status->carrier);
}

static void link_config(const uint64_t *time, enum
    nrf_modem_dect_phy_err err) {}
static void time_get(const uint64_t *time, enum
    nrf_modem_dect_phy_err err)
{
    if (err == 0) {
        latest_modem_time = *time;
    } else {
        LOG_ERR("Failed_to_get_modem_time:%d", err);
    }
    k_sem_give(&operation_sem);
}

static void capability_get(const uint64_t *time, enum
    nrf_modem_dect_phy_err err,
    const struct nrf_modem_dect_phy_capability
        *capability) {}

static void stf_cover_seq_control(const uint64_t *time, enum
    nrf_modem_dect_phy_err err)
{
    LOG_WRN("Unexpected_STF_cover_sequence_control_callback.");
}

static void rx_stop(const uint64_t *time, enum nrf_modem_dect_phy_err
    err, uint32_t handle)
{
    LOG_DBG("RX_stopped");
    k_sem_give(&operation_sem);
}

static struct nrf_modem_dect_phy_callbacks dect_phy_callbacks = {
    .init = init,
    .deinit = deinit,
    .op_complete = op_complete,
    .rx_stop = rx_stop,
    .pcc = pcc,
    .pcc_crc_err = pcc_crc_err,
    .pdc = pdc,
    .pdc_crc_err = pdc_crc_err,
    .rssi = rssi,
    .link_config = link_config,
    .time_get = time_get,
    .capability_get = capability_get,

```

```

        .stf_cover_seq_control = stf_cover_seq_control,
    };

static int transmit(uint32_t handle, void *data, size_t data_len)
{
    struct phy_ctrl_field_common header = {
        .header_format = 0x0,
        .packet_length_type = 0x0,
        .packet_length = 0x01,
        .short_network_id = (CONFIG_NETWORK_ID & 0xff),
        .transmitter_id_hi = (device_id >> 8),
        .transmitter_id_lo = (device_id & 0xff),
        .transmit_power = 15,
        .reserved = 0,
        .df_mcs = 1,
    };

    struct nrf_modem_dect_phy_tx_params tx_op_params = {
        .start_time = 0,
        .handle = handle,
        .network_id = CONFIG_NETWORK_ID,
        .phy_type = 0,
        .lbt_rssi_threshold_max = 0,
        .carrier = CONFIG_CARRIER,
        .lbt_period = 0,
        .phy_header = (union nrf_modem_dect_phy_hdr *)&header,
        .data = data,
        .data_size = data_len,
    };

    return nrf_modem_dect_phy_tx(&tx_op_params);
}

static int receive(uint32_t handle)
{
    struct nrf_modem_dect_phy_rx_params rx_op_params = {
        .start_time = 0,
        .handle = handle,
        .network_id = CONFIG_NETWORK_ID,
        .mode = NRF_MODEM_DECT_PHY_RX_MODE_SINGLE_SHOT,
        .rssi_interval = NRF_MODEM_DECT_PHY_RSSI_INTERVAL_OFF,
        .link_id = NRF_MODEM_DECT_PHY_LINK_UNSPECIFIED,
        .rssi_level = -60,
        .carrier = CONFIG_CARRIER,
        .duration = CONFIG_RX_PERIOD_S_mod * MSEC_PER_SEC *
                    NRF_MODEM_DECT_MODEM_TIME_TICK_RATE_KHZ,
        .filter.short_network_id = CONFIG_NETWORK_ID & 0xff,
        .filter.is_short_network_id_used = 0,
        .filter.receiver_identity = 0,
    };

    return nrf_modem_dect_phy_rx(&rx_op_params);
}

```

```

}

int main(void)
{
    int err;
    uint32_t tx_handle = 0;
    uint32_t rx_handle = 1;

    LOG_INF("RTT_Test:_Board_A_(Initiator)");

    err = nrf_modem_lib_init();
    if (err) {
        LOG_ERR("Modem_init_failed:_%d", err);
        return err;
    }

    err = nrf_modem_dect_phy_callback_set(&dect_phy_callbacks);
    if (err) return err;

    struct nrf_modem_dect_phy_init_params init_params = {
        .harq_rx_expiry_time_us = 50000000,
        .harq_rx_process_count = 4,
    };

    err = nrf_modem_dect_phy_init(&init_params);
    if (err) return err;

    k_sem_take(&operation_sem, K_FOREVER);
    if (exit) return -EIO;

    hwinfo_get_device_id((void *)&device_id, sizeof(device_id));
    LOG_INF("Device_ID:_%d", device_id);

    struct rtt_packet pkt;
    uint8_t tx_buf[DATA_LEN_MAX];
    uint32_t pkt_id = 0;

    while (1) {
        // Get modem time
        err = nrf_modem_dect_phy_time_get();
        if (err) return err;
        k_sem_take(&operation_sem, K_FOREVER);

        // Prepare and send packet
        pkt.pkt_id = pkt_id++;
        pkt.tx_time_ticks = latest_modem_time;
        memcpy(tx_buf, &pkt, sizeof(pkt));

        err = transmit(tx_handle, tx_buf, sizeof(pkt));
        if (err) {
            LOG_ERR("TX_failed:_%d", err);
        } else {

```

```

        LOG_INF("Sent_RTT_pkt_%d_at_tick_" PRIu64, pkt.pkt_id,
                pkt.tx_time_ticks);
    }
    k_sem_take(&operation_sem, K_FOREVER);

    // Start RX
    err = receive(rx_handle);
    if (err) {
        LOG_ERR("RX_start_failed:_%d", err);
        return err;
    }

    k_sem_take(&operation_sem, K_FOREVER);

    k_sleep(K_MSEC(2000)); // Delay before next cycle
}

return 0;
}

```

Appendix 3 RTT measurement program : Server

```
// Board B      RTT Echo Node

#include <string.h>
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include <nrf_modem_dect_phy.h>
#include <modem/nrf_modem_lib.h>
#include <zephyr/drivers/hwinfo.h>

#define DATA_LEN_MAX 32
#define CONFIG_RX_PERIOD_S_mod 5
#define RX_TIMEOUT_MS 3000

K_SEM_DEFINE(operation_sem, 0, 1);

static bool exit;
static uint16_t device_id;

static struct rtt_packet {
    uint32_t pkt_id;
    uint64_t tx_time_ticks;
} echo_pkt;

static struct nrf_modem_dect_phy_tx_params saved_tx_params;

static struct phy_ctrl_field_common {
    uint32_t packet_length : 4;
    uint32_t packet_length_type : 1;
    uint32_t header_format : 3;
    uint32_t short_network_id : 8;
    uint32_t transmitter_id_hi : 8;
    uint32_t transmitter_id_lo : 8;
    uint32_t df_mcs : 3;
    uint32_t reserved : 1;
    uint32_t transmit_power : 4;
    uint32_t pad : 24;
} echo_header = {
    .header_format = 0x0,
    .packet_length_type = 0x0,
    .packet_length = 0x01,
    .short_network_id = (CONFIG_NETWORK_ID & 0xff),
    .transmitter_id_hi = 0,
    .transmitter_id_lo = 0,
    .transmit_power = CONFIG_TX_POWER,
    .reserved = 0,
    .df_mcs = CONFIG_MCS,
};

static bool echo_pending = false;
static int64_t last_rx_time = 0;
```

```

LOG_MODULE_REGISTER(app);
BUILD_ASSERT(CONFIG_CARRIER, "Carrier_must_be_configured_correctly.")
;

/* Callback: Modem initialization */
static void init(const uint64_t *time, int16_t temp, enum
    nrf_modem_dect_phy_err err,
    const struct nrf_modem_dect_phy_modem_cfg *cfg)
{
    if (err) {
        LOG_ERR("Init_failed,_err_%d", err);
        exit = true;
        return;
    }
    k_sem_give(&operation_sem);
}

/* Callback: General operation complete */
static void op_complete(const uint64_t *time, int16_t temp,
    enum nrf_modem_dect_phy_err err, uint32_t
    handle)
{
    k_sem_give(&operation_sem);
}

/* Callback: Deinitialization */
static void deinit(const uint64_t *time, enum nrf_modem_dect_phy_err
    err)
{
    if (err) {
        LOG_ERR("Deinit_failed,_err_%d", err);
        return;
    }
    k_sem_give(&operation_sem);
}

/* Callback: Control and data reception */
static void pdc(const uint64_t *time,
    const struct nrf_modem_dect_phy_rx_pdc_status *status
    ,
    const void *data, uint32_t len)
{
    memcpy(&echo_pkt, data, sizeof(echo_pkt));

    saved_tx_params = (struct nrf_modem_dect_phy_tx_params){
        .start_time = 0,
        .handle = 0,
        .network_id = CONFIG_NETWORK_ID,
        .phy_type = 0,
        .lbt_rssi_threshold_max = 0,
        .carrier = CONFIG_CARRIER,
    };
}

```

```

        .lbt_period = 0,
        .phy_header = (union nrf_modem_dect_phy_hdr *)&echo_header,
        .data = (uint8_t *)&echo_pkt,
        .data_size = sizeof(echo_pkt),
    };

    echo_pending = true;
    last_rx_time = k_uptime_get();
}

static void pdc_crc_err(const uint64_t *time,
                       const struct
                           nrf_modem_dect_phy_rx_pdc_crc_failure *
                           crc_failure) {}

static void pcc(const uint64_t *time,
                const struct nrf_modem_dect_phy_rx_pcc_status *status
                ,
                const union nrf_modem_dect_phy_hdr *hdr) {}

static void pcc_crc_err(const uint64_t *time,
                       const struct
                           nrf_modem_dect_phy_rx_pcc_crc_failure *
                           crc_failure) {}

static void rssi(const uint64_t *time,
                 const struct nrf_modem_dect_phy_rssi_meas *status)
    {}

static void link_config(const uint64_t *time, enum
    nrf_modem_dect_phy_err err) {}
static void time_get(const uint64_t *time, enum
    nrf_modem_dect_phy_err err)
{
    k_sem_give(&operation_sem);
}

static void capability_get(const uint64_t *time, enum
    nrf_modem_dect_phy_err err,
                       const struct nrf_modem_dect_phy_capability
                           *capability) {}

static void stf_cover_seq_control(const uint64_t *time, enum
    nrf_modem_dect_phy_err err) {}

static void rx_stop(const uint64_t *time, enum nrf_modem_dect_phy_err
    err, uint32_t handle)
{
    k_sem_give(&operation_sem);
}

/* Callback registration */

```

```

static struct nrf_modem_dect_phy_callbacks dect_phy_callbacks = {
    .init = init,
    .deinit = deinit,
    .op_complete = op_complete,
    .rx_stop = rx_stop,
    .pcc = pcc,
    .pcc_crc_err = pcc_crc_err,
    .pdc = pdc,
    .pdc_crc_err = pdc_crc_err,
    .rssi = rssi,
    .link_config = link_config,
    .time_get = time_get,
    .capability_get = capability_get,
    .stf_cover_seq_control = stf_cover_seq_control,
};

/* Start reception in continuous mode */
static int receive(uint32_t handle)
{
    struct nrf_modem_dect_phy_rx_params rx_op_params = {
        .start_time = 0,
        .handle = handle,
        .network_id = CONFIG_NETWORK_ID,
        .mode = NRF_MODEM_DECT_PHY_RX_MODE_CONTINUOUS,
        .rssi_interval = NRF_MODEM_DECT_PHY_RSSI_INTERVAL_OFF,
        .link_id = NRF_MODEM_DECT_PHY_LINK_UNSPECIFIED,
        .rssi_level = -60,
        .carrier = CONFIG_CARRIER,
        .duration = CONFIG_RX_PERIOD_S_mod * MSEC_PER_SEC *
                    NRF_MODEM_DECT_MODEM_TIME_TICK_RATE_KHZ,
        .filter.short_network_id = CONFIG_NETWORK_ID & 0xff,
        .filter.is_short_network_id_used = 0,
        .filter.receiver_identity = 0,
    };

    return nrf_modem_dect_phy_rx(&rx_op_params);
}

int main(void)
{
    int err;

    LOG_INF("Board_B: Echo_node_started");

    err = nrf_modem_lib_init();
    if (err) return err;

    err = nrf_modem_dect_phy_callback_set(&dect_phy_callbacks);
    if (err) return err;

    struct nrf_modem_dect_phy_init_params init_params = {
        .harq_rx_expiry_time_us = 5000000,
    };

```

```

        .harq_rx_process_count = 4,
};

err = nrf_modem_dect_phy_init(&init_params);
if (err) return err;

k_sem_take(&operation_sem, K_FOREVER);
if (exit) return -EIO;

hwinf_get_device_id((void *)&device_id, sizeof(device_id));
LOG_INF("Device_ID: %d", device_id);

echo_header.transmitter_id_hi = (device_id >> 8);
echo_header.transmitter_id_lo = (device_id & 0xff);
last_rx_time = k_uptime_get();

err = receive(1);
if (err) return err;
LOG_INF("Ready to receive and echo RTT packets");

while (1) {
    if (echo_pending) {
        echo_pending = false;

        err = nrf_modem_dect_phy_rx_stop(1);
        k_sem_take(&operation_sem, K_FOREVER);

        k_busy_wait(170); // Small delay before echoing

        err = nrf_modem_dect_phy_tx(&saved_tx_params);
        k_sem_take(&operation_sem, K_FOREVER);

        LOG_INF("Echoed packet ID = %d, TX tick = %" PRIu64,
                echo_pkt.pkt_id, echo_pkt.tx_time_ticks);

        k_sleep(K_MSEC(6)); // Ensure RX restarts correctly
        err = receive(1);
        if (err) return err;
    }

    if (k_uptime_get() - last_rx_time > RX_TIMEOUT_MS) {
        LOG_WRN("No packets recently. Restarting RX...");
        nrf_modem_dect_phy_rx_stop(1);
        k_sem_take(&operation_sem, K_FOREVER);
        k_sleep(K_MSEC(10));
        err = receive(1);
        if (!err) last_rx_time = k_uptime_get();
    }
}

return 0;
}

```

Appendix 4 Throughput measurement program : TX

```
// DECT NR+ Throughput Measurement Application

#include <string.h>
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include <nrf_modem_dect_phy.h>
#include <modem/nrf_modem_lib.h>
#include <zephyr/drivers/hwinfo.h>

LOG_MODULE_REGISTER(app);

BUILD_ASSERT(CONFIG_CARRIER, "Carrier_must_be_configured_properly");

#define DATA_LEN_MAX 32
#define MAX_MCS 5
#define MAX_SLOTS 16

// Transmission configuration
uint8_t tx_power = 15;
uint8_t tx_slots = 7;
uint8_t tx_mcs = 1;
uint32_t N = 101;

uint16_t TBS = 0;
uint64_t time_t1 = 0, time_t2 = 0;
uint64_t pkt_counter = 0;
uint32_t tx_byteCount = 0;
bool tx_1stpkt = true;
uint64_t ticks = 0;

static bool exit;
static uint16_t device_id;

// TBS lookup table
static const uint16_t tbs_lookup[MAX_MCS][MAX_SLOTS] = {
    { 0, 136, 264, 400, 536, 664, 792, 920, 1064, 1192, 1320, 1448,
      1576, 1704, 1864, 1992 },
    { 32, 296, 552, 824, 1096, 1352, 1608, 1864, 2104, 2360, 2616,
      2872, 3128, 3384, 3704, 3960 },
    { 56, 456, 856, 1256, 1640, 2024, 2360, 2744, 3192, 3576, 3960,
      4320, 4768, 5152, 5536, 0 },
    { 88, 616, 1128, 1672, 2168, 2680, 3192, 3704, 4256, 4768, 5280,
      0, 0, 0, 0, 0 },
    { 144, 936, 1736, 2488, 3256, 4024, 4832, 5600, 0, 0, 0, 0, 0, 0,
      0, 0 }
};

uint16_t get_tbs(uint8_t mcs, uint8_t subslot_index) {
    if (mcs >= MAX_MCS || subslot_index >= MAX_SLOTS) return 0;
    return tbs_lookup[mcs][subslot_index];
}
```

```

}

struct phy_ctrl_field_common {
    uint32_t packet_length : 4;
    uint32_t packet_length_type : 1;
    uint32_t header_format : 3;
    uint32_t short_network_id : 8;
    uint32_t transmitter_id_hi : 8;
    uint32_t transmitter_id_lo : 8;
    uint32_t df_mcs : 3;
    uint32_t reserved : 1;
    uint32_t transmit_power : 4;
    uint32_t pad : 24;
};

K_SEM_DEFINE(operation_sem, 0, 1);

static void init(const uint64_t *time, int16_t temp, enum
    nrf_modem_dect_phy_err err,
    const struct nrf_modem_dect_phy_modem_cfg *cfg)
{
    if (err) {
        LOG_ERR("Init_failed:_%d", err);
        exit = true;
        return;
    }
    k_sem_give(&operation_sem);
}

static void deinit(const uint64_t *time, enum nrf_modem_dect_phy_err
    err)
{
    if (err) {
        LOG_ERR("Deinit_failed:_%d", err);
        return;
    }
    k_sem_give(&operation_sem);
}

static void op_complete(const uint64_t *time, int16_t temperature,
    enum nrf_modem_dect_phy_err err, uint32_t
    handle)
{
    pkt_counter++;
    if (tx_1stpkt) {
        time_t1 = *time;
        tx_1stpkt = false;
    }
    if (pkt_counter >= N) {
        time_t2 = *time;
        LOG_DBG("TX_complete_at_pkt_%d", pkt_counter - 1);
    }
}

```

```

        k_sem_give(&operation_sem);
    }

    static void rx_stop(const uint64_t *time, enum nrf_modem_dect_phy_err
        err, uint32_t handle)
    {
        LOG_DBG("RX_stopped_at%" PRIu64, *time);
        k_sem_give(&operation_sem);
    }

    static void pcc(const uint64_t *time,
        const struct nrf_modem_dect_phy_rx_pcc_status *status
        ,
        const union nrf_modem_dect_phy_hdr *hdr)
    {
        struct phy_ctrl_field_common *header = (struct
            phy_ctrl_field_common *)hdr->type_1;
        LOG_INF("Received_header_from_device_ID%" PRIu64,
            (header->transmitter_id_hi << 8) | header->
            transmitter_id_lo);
    }

    static void pcc_crc_err(const uint64_t *time,
        const struct
            nrf_modem_dect_phy_rx_pcc_crc_failure *
            crc_failure) {}

    static void pdc(const uint64_t *time,
        const struct nrf_modem_dect_phy_rx_pdc_status *status
        ,
        const void *data, uint32_t len)
    {
        LOG_INF("Received_data_(RSSI:_%d.%d):_%s",
            (status->rssi_2 / 2), (status->rssi_2 & 0x1) * 5, (
            char *)data);
    }

    static void pdc_crc_err(const uint64_t *time,
        const struct
            nrf_modem_dect_phy_rx_pdc_crc_failure *
            crc_failure) {}

    static void rssi(const uint64_t *time, const struct
        nrf_modem_dect_phy_rssi_meas *status) {}

    static void link_config(const uint64_t *time, enum
        nrf_modem_dect_phy_err err) {}
    static void time_get(const uint64_t *time, enum
        nrf_modem_dect_phy_err err) {}
    static void capability_get(const uint64_t *time, enum
        nrf_modem_dect_phy_err err,
        const struct nrf_modem_dect_phy_capability

```

```

                                *capability) {}
static void stf_cover_seq_control(const uint64_t *time, enum
    nrf_modem_dect_phy_err err) {}

static struct nrf_modem_dect_phy_callbacks dect_phy_callbacks = {
    .init = init,
    .deinit = deinit,
    .op_complete = op_complete,
    .rx_stop = rx_stop,
    .pcc = pcc,
    .pcc_crc_err = pcc_crc_err,
    .pdc = pdc,
    .pdc_crc_err = pdc_crc_err,
    .rssi = rssi,
    .link_config = link_config,
    .time_get = time_get,
    .capability_get = capability_get,
    .stf_cover_seq_control = stf_cover_seq_control,
};

static struct nrf_modem_dect_phy_init_params dect_phy_init_params = {
    .harq_rx_expiry_time_us = 5000000,
    .harq_rx_process_count = 4,
};

static int transmit(uint32_t handle, void *data, size_t data_len)
{
    struct phy_ctrl_field_common header = {
        .header_format = 0x0,
        .packet_length_type = 0x00,
        .packet_length = tx_slots,
        .short_network_id = (CONFIG_NETWORK_ID & 0xff),
        .transmitter_id_hi = (device_id >> 8),
        .transmitter_id_lo = (device_id & 0xff),
        .transmit_power = tx_power,
        .reserved = 0,
        .df_mcs = tx_mcs,
    };

    struct nrf_modem_dect_phy_tx_params tx_op_params = {
        .start_time = 0,
        .handle = handle,
        .network_id = CONFIG_NETWORK_ID,
        .phy_type = 0,
        .lbt_rssi_threshold_max = 0,
        .carrier = CONFIG_CARRIER,
        .lbt_period = 0,
        .phy_header = (union nrf_modem_dect_phy_hdr *)&header,
        .data = data,
        .data_size = data_len,
    };
};

```

```

        return nrf_modem_dect_phy_tx(&tx_op_params);
    }

int main(void)
{
    int err;
    uint8_t tx_buf[DATA_LEN_MAX];
    const char *payload = "<<<START>>>
        ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789...<<<END>>>";
    uint16_t tbs_bytes;

    LOG_INF("DECT_NR+_PHY_Throughput_Test_Started");

    err = nrf_modem_lib_init();
    if (err) return err;

    err = nrf_modem_dect_phy_callback_set(&dect_phy_callbacks);
    if (err) return err;

    err = nrf_modem_dect_phy_init(&dect_phy_init_params);
    if (err) return err;

    k_sem_take(&operation_sem, K_FOREVER);
    if (exit) return -EIO;

    hwinfo_get_device_id((void *)&device_id, sizeof(device_id));
    LOG_INF("Device_ID:_%d", device_id);

    err = nrf_modem_dect_phy_capability_get();
    if (err) LOG_ERR("Capability_query_failed");

    err = nrf_modem_dect_phy_time_get();
    if (err) LOG_ERR("Time_get_failed");

    TBS = get_tbs(tx_mcs, tx_slots);
    tbs_bytes = TBS / 8;

    for (int p = 0; p < N; p++) {
        err = transmit(0, (void *)payload, tbs_bytes);
        if (err) LOG_ERR("TX_failed_at_pkt_%d", p);
        tx_byteCount += tbs_bytes;
        k_sem_take(&operation_sem, K_FOREVER);
    }

    if (time_t2 > time_t1) {
        ticks = time_t2 - time_t1;
    } else {
        ticks = (UINT64_MAX - time_t1) + time_t2 + 1;
    }

    LOG_INF("TX_bytes_(excluding_last_pkt):_%d", tx_byteCount -

```

```

        tbs_bytes);
LOG_INF("Ticks_used:_%d" PRIu64, ticks);

float rate = (tbs_bytes * (N - 1) * 8.0f) / ((float)ticks /
        NRF_MODEM_DECT_MODEM_TIME_TICK_RATE_KHZ);
LOG_INF("Data_Rate:_%d.%03d_kbit/s", (int)rate, (int)((rate -
        (int)rate) * 1000));

k_sleep(K_SECONDS(5));

err = nrf_modem_dect_phy_deinit();
if (err) return err;

k_sem_take(&operation_sem, K_FOREVER);

err = nrf_modem_lib_shutdown();
if (err) return err;

LOG_INF("Shutdown_complete");
return 0;
}

```

Appendix 5 Throughput measurement program : RX

```
// DECT NR+ Throughput Receiver Application

#include <string.h>
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include <nrf_modem_dect_phy.h>
#include <modem/nrf_modem_lib.h>
#include <zephyr/drivers/hwinfo.h>

LOG_MODULE_REGISTER(app);

BUILD_ASSERT(CONFIG_CARRIER, "Carrier_must_be_configured_properly");

#define DATA_LEN_MAX 32
#define N 101 // Number of packets to receive

uint8_t tx_power = 7;
uint32_t rxpkt_count = 0;
uint64_t rxByte_count = 0;
uint64_t t1 = 0, t2 = 0;
bool rx_1st_pkt = true;
bool rx_done = false;

static bool exit;
static uint16_t device_id;

struct phy_ctrl_field_common {
    uint32_t packet_length : 4;
    uint32_t packet_length_type : 1;
    uint32_t header_format : 3;
    uint32_t short_network_id : 8;
    uint32_t transmitter_id_hi : 8;
    uint32_t transmitter_id_lo : 8;
    uint32_t df_mcs : 3;
    uint32_t reserved : 1;
    uint32_t transmit_power : 4;
    uint32_t pad : 24;
};

K_SEM_DEFINE(operation_sem, 0, 1);

static void init(const uint64_t *time, int16_t temp, enum
    nrf_modem_dect_phy_err err,
    const struct nrf_modem_dect_phy_modem_cfg *cfg)
{
    if (err) {
        LOG_ERR("Init_failed:_%d", err);
        exit = true;
        return;
    }
}
```

```

        k_sem_give(&operation_sem);
    }

    static void deinit(const uint64_t *time, enum nrf_modem_dect_phy_err
        err)
    {
        if (err) {
            LOG_ERR("Deinit failed: %d", err);
            return;
        }
        k_sem_give(&operation_sem);
    }

    static void op_complete(const uint64_t *time, int16_t temperature,
        enum nrf_modem_dect_phy_err err, uint32_t
        handle)
    {
        k_sem_give(&operation_sem);
    }

    static void rx_stop(const uint64_t *time, enum nrf_modem_dect_phy_err
        err, uint32_t handle)
    {
        k_sem_give(&operation_sem);
    }

    static void pcc(const uint64_t *time,
        const struct nrf_modem_dect_phy_rx_pcc_status *status
        ,
        const union nrf_modem_dect_phy_hdr *hdr) {}

    static void pcc_crc_err(const uint64_t *time,
        const struct
            nrf_modem_dect_phy_rx_pcc_crc_failure *
            crc_failure) {}

    static void pdc(const uint64_t *time,
        const struct nrf_modem_dect_phy_rx_pdc_status *status
        ,
        const void *data, uint32_t len)
    {
        rxpkt_count++;

        if (rx_1st_pkt) {
            t1 = *time;
            rx_1st_pkt = false;
            rxByte_count = 0;
        } else {
            rxByte_count += len;
        }

        if (rxpkt_count >= N) {

```

```

        t2 = *time;
        rx_done = true;
        LOG_INF("Total_RX_Data:_%d" PRIu64 "Bytes",
                rxByte_count);
    }
}

static void pdc_crc_err(const uint64_t *time,
                       const struct
                           nrf_modem_dect_phy_rx_pdc_crc_failure *
                           crc_failure)
{
    LOG_ERR("PDC_CRC_error_at_%d" PRIu64, *time);

    if (crc_failure->rssi_2 !=
        NRF_MODEM_DECT_PHY_RSSI2_NOT_MEASURED) {
        LOG_ERR("RSSI:_%d.%d_dBm", crc_failure->rssi_2 / 2,
                (crc_failure->rssi_2 & 1) * 5);
    }
    if (crc_failure->snr != NRF_MODEM_DECT_PHY_SNR_NOT_MEASURED)
    {
        LOG_ERR("SNR:_%d.%02d_dB", crc_failure->snr >> 2, (
                crc_failure->snr & 3) * 25);
    }
}

static void rssi(const uint64_t *time, const struct
    nrf_modem_dect_phy_rssi_meas *status) {}
static void link_config(const uint64_t *time, enum
    nrf_modem_dect_phy_err err) {}
static void time_get(const uint64_t *time, enum
    nrf_modem_dect_phy_err err) {}
static void capability_get(const uint64_t *time, enum
    nrf_modem_dect_phy_err err,
                           const struct nrf_modem_dect_phy_capability
                           *capability) {}
static void stf_cover_seq_control(const uint64_t *time, enum
    nrf_modem_dect_phy_err err) {}

static struct nrf_modem_dect_phy_callbacks dect_phy_callbacks = {
    .init = init,
    .deinit = deinit,
    .op_complete = op_complete,
    .rx_stop = rx_stop,
    .pcc = pcc,
    .pcc_crc_err = pcc_crc_err,
    .pdc = pdc,
    .pdc_crc_err = pdc_crc_err,
    .rssi = rssi,
    .link_config = link_config,
    .time_get = time_get,
    .capability_get = capability_get,

```

```

        .stf_cover_seq_control = stf_cover_seq_control,
};

static struct nrf_modem_dect_phy_init_params dect_phy_init_params = {
    .harq_rx_expiry_time_us = 5000000,
    .harq_rx_process_count = 4,
};

static int receive(uint32_t handle)
{
    struct nrf_modem_dect_phy_rx_params rx_op_params = {
        .start_time = 0,
        .handle = handle,
        .network_id = CONFIG_NETWORK_ID,
        .mode = NRF_MODEM_DECT_PHY_RX_MODE_CONTINUOUS,
        .rssi_interval = NRF_MODEM_DECT_PHY_RSSI_INTERVAL_OFF,
        ,
        .link_id = NRF_MODEM_DECT_PHY_LINK_UNSPECIFIED,
        .rssi_level = -60,
        .carrier = CONFIG_CARRIER,
        .duration = 60 * MSEC_PER_SEC *
            NRF_MODEM_DECT_MODEM_TIME_TICK_RATE_KHZ,
        .filter.short_network_id = CONFIG_NETWORK_ID & 0xff,
        .filter.is_short_network_id_used = 1,
        .filter.receiver_identity = 0,
    };

    return nrf_modem_dect_phy_rx(&rx_op_params);
}

int main(void)
{
    int err;

    LOG_INF("DECT_NRF+PHY_Receiver_Started");

    err = nrf_modem_lib_init();
    if (err) return err;

    err = nrf_modem_dect_phy_callback_set(&dect_phy_callbacks);
    if (err) return err;

    err = nrf_modem_dect_phy_init(&dect_phy_init_params);
    if (err) return err;

    k_sem_take(&operation_sem, K_FOREVER);
    if (exit) return -EIO;

    hwinfo_get_device_id((void *)&device_id, sizeof(device_id));
    LOG_INF("Device_ID: %d", device_id);

    err = nrf_modem_dect_phy_capability_get();

```

```

    if (err) LOG_ERR("Capability_query_failed");

    err = receive(1);
    if (err) {
        LOG_ERR("Reception_failed:_%d", err);
    } else {
        LOG_INF("Receiver_started");
    }

    while (1) {
        if (rx_done) {
            uint32_t ticks = t2 - t1;
            LOG_INF("RX_Duration:_%d_ticks", ticks);

            float rate = rxByte_count * 8.0f / ((float)
                ticks /
                NRF_MODEM_DECT_MODEM_TIME_TICK_RATE_KHZ);
            int rate_int = (int)rate;
            int rate_frac = (int)((rate - rate_int) *
                1000);

            LOG_INF("Data_Rate:_%d.%03d_kbit/s", rate_int
                , rate_frac);
            break;
        }
        k_sleep(K_SECONDS(2));
    }

    LOG_INF("Shutting_down");

    err = nrf_modem_dect_phy_deinit();
    if (err) return err;
    k_sem_take(&operation_sem, K_FOREVER);

    err = nrf_modem_lib_shutdown();
    if (err) return err;

    LOG_INF("Receiver_exit_complete");
    return 0;
}

```